TWIST: Private Tenant-Dedicated LLM Serving via Joint Model-Token Transformation

Extended Version

Qizhi Zhang*
ByteDance
Shenzhen, China
zhangqizhi.zqz@bytedance.com

Yufei Chen*
City University of Hong Kong
Hong Kong, China
yufeichen8@cityu.edu.hk

Yu Lin*
ByteDance
Shenzhen, China
linyu.09@bytedance.com

Wenqiang Ruan
ByteDance
Shenzhen, China
ruanwenqiang@bytedance.com

Quanwei Cai
ByteDance
Beijing, China
caiquanwei@bytedance.com

Daode Zhang
ByteDance
Beijing, China
zhangdaode@bytedance.com

Longxiang Wang
City University of Hong Kong
Hong Kong, China
longxiang.wang@my.cityu.edu.hk

Jue Hong
ByteDance
Shenzhen, China
tanzhuo.107@bytedance.com

Ye Wu†
ByteDance
Shenzhen, China
wuye.2020@bytedance.com

Cong Wang†
City University of Hong Kong
Hong Kong, China
congwang@cityu.edu.hk

Abstract

Organizations increasingly host open-weight large language models (LLMs) on dedicated cloud instances to avoid the opaque data control imposed by closed-source APIs and the burden of costly local infrastructure. However, this deployment paradigm inadvertently exposes sensitive tenant prompts and outputs to service providers. Existing protection routes do not fit this setting well in practice: cryptographic inference incurs prohibitive overheads; Trusted Execution Environments (TEEs) rely on specialized hardware that is difficult to deploy consistently across heterogeneous legacy accelerators; and traditional data-only obfuscation methods, such as token replacement, require substantial perturbation to achieve meaningful privacy, often causing severe utility degradation.

To address this gap, we present TWIST, a practical system for private open-weight LLM serving on dedicated cloud instances. Rather than perturbing input tokens alone, TWIST applies a shared, structure-preserving transformation to both the model weights and input tokens. This coupled design extends protection beyond data-only obfuscation and addresses the traditional utility bottleneck. We analyze leakage to an honest-but-curious provider using Rényi-metric differential privacy (RmDP), and show a smaller privacy budget than a data-only baseline with the same online mechanism.

Practically, we implement TWIST for modern dense and Mixture-of-Experts (MoE) architectures, including attention, FFN, and MoE routing modules, without disrupting standard serving pipelines. Evaluations on the 671B-parameter DeepSeek-V3.1-Terminus model show that TWIST reduces token recovery rate to under 5% against advanced inversion attacks, while incurring only 0.0% - 3.5% accuracy loss and <10% inference speed loss, offering a practical solution for private tenant-dedicated LLM serving.

CCS Concepts

• **Security and privacy** → **Privacy-preserving protocols**; • **Computing methodologies** → *Machine learning*.

Keywords

Privacy-preserving LLM inference, differential privacy, prompt privacy

1 Introduction

Deploying large language models forces organizations to trade off control against cost. Commercial model APIs offer capacity at scale, but every prompt leaves the organization, and privacy relies largely on provider-side protections rather than tenant-controlled technical guarantees. Self-hosting keeps the data path within the organization, yet serving modern open-weight models such as Llama [63], DeepSeek-V3 [14], and Qwen3 [72] requires substantial compute resources and operational expertise that many organizations cannot sustain. This tension motivates a third deployment model: tenant-dedicated managed serving, commonly referred to as Bring Your Own Model (BYOM) [3, 21, 29, 45], where the tenant provides an open-weight model checkpoint and a cloud provider performs inference on rented GPUs using standard serving engines such as vLLM [33] or SGLang [78]. BYOM restores model ownership and deployment flexibility to the tenant, but not execution privacy: while a tenant-side client may remain local, the provider still performs

*These authors contributed equally to this work.
†Corresponding authors: Ye Wu and Cong Wang.

the autoregressive forward pass and observes every prompt and generated token in plaintext, even when faithfully following the protocol. That exposure is unacceptable for sensitive workloads such as patient records, legal documents, proprietary code, and other identifying data.

Protecting such workloads requires outsourced inference without exposing the underlying data to the provider. Existing privacy-preserving inference approaches, however, make assumptions or incur costs that do not align well with tenant-dedicated BYOM serving: (1) *Cryptographic inference* performs model evaluation over encrypted or secret-shared tensors, substituting standard Transformer operations with MPC or homomorphic encryption protocols. Cryptographic inference provides rigorous privacy guarantees, but tensor multiplications, attention nonlinearities, and activations become communication-intensive cryptographic operations, which severely bottleneck autoregressive token generation [25, 34, 41]. (2) *TEE-based inference* relies on hardware-enforced isolation for plaintext execution. PipeLLM uses confidential GPUs [59], while ccAI protects accelerators through a PCIe security controller [68]. CPU-TEE approaches place protected computation in CPU enclaves and delegate other operations to GPUs [48, 64, 77]. Their protection therefore depends on trusted hardware support that may not be available across the heterogeneous accelerators used for tenant-dedicated serving. (3) *Data-only obfuscation* and split learning are lower-overhead alternatives. The former randomizes tokens or perturbs input embeddings [16, 44, 62]. The latter runs early layers on the client and sends activations to the provider [24, 67]. In both cases, the provider-side model remains in its original representation space, leaving transmitted representations potentially vulnerable to inversion. Prior attacks have demonstrated this risk for both obfuscated embeddings [40] and deeper internal states [15]. Split-learning studies further demonstrate training-stage recovery, including active manipulation of training [20, 50].

BYOM gives the tenant another design option: transforming the model checkpoint before deployment.

We present TWIST (*Tenant-dedicated Weight-Input Space Transformation*), a system for private tenant-dedicated LLM serving that jointly transforms the token space and model weights into a shared coordinate system. By transforming the model and input together, TWIST keeps the transformed tokens consistent with the transformed model checkpoint.

In practice, the tenant first applies the joint transformation to the model checkpoint before deployment. Then, at inference time, the client sends provider-visible token IDs, the provider performs standard autoregressive decoding on the transformed model checkpoint, and the client maps returned token IDs back to plaintext.

A usable transformation must hide token-model correspondences without breaking the operations required by standard inference engines. Simple token permutations can be deanonymized through embedding fingerprints, while arbitrary hidden-space transformations break Transformer operations such as attention softmax, nonlinear FFNs, and residual additions. TWIST instead tailors the transformation to the algebra of each Transformer module. We identify invertible transformations that preserve the required operations and cancel within the forward pass. Specifically, we apply a token permutation π (a vocabulary-level remapping secured by calibrated noise and masking) at the token boundaries, and invertible weight transformations from a family G within internal modules, restricted to what each module (e.g., Attention or MoE) can natively preserve. Jointly transforming weights and tokens reduces reliance on input perturbation for privacy.

We then quantify what the provider learns from the transformed checkpoint and request tokens. In our setting, the provider observes a one-time offline checkpoint release and repeated transformed input tokens, while the secrets of interest are the token permutation π and the plaintext token sequence.

Classical differential privacy (DP) [19] is built around neighboring datasets, whereas our secrets are token permutations and sequences. For the Gaussian randomization used in the offline release, the standard (ε, δ)-DP view composes loosely, while Rényi differential privacy (RDP) [46] gives a tighter treatment of Gaussian composition. Because the secrets are structured objects rather than neighboring records, and the offline release uses Gaussian randomization, we analyze the joint mechanism under **Rényi-metric differential privacy (RmDP)**. This formulation uses a metric over secrets [6] and distance-scaled Rényi bounds related to concentrated geo-privacy [36], as specified in Definition 5.1. Within this framework, we derive a composed privacy bound for the joint mechanism by composing the offline checkpoint release with the request-level token-sequence mechanism, yielding a distance-scaled guarantee on plaintext-sequence recovery (Theorem 5.4). Under this analysis, the resulting privacy budget is smaller than that of a data-only baseline using the same online token-perturbation mechanism.

We evaluate TWIST across eight open-weight LLMs, including dense and MoE architectures up to the 671B-parameter DeepSeek-V3.1-Terminus deployment [11, 14], under strong provider-side recovery attacks and online serving overhead measurements. On Qwen2.5-14B-Instruct, TWIST stays close to plaintext utility while reducing token recovery far below the evaluated data-only baselines, showing that joint model-token transformation improves the privacy–utility trade-off. On DeepSeek-V3.1-Terminus, TWIST achieves 4.80% text token recovery success ratio (TTRSR), with modest utility loss and single-digit online overhead.

This work makes three contributions:

- **Joint model-token transformation.** We introduce a mechanism that uses offline model access to transform weights and tokens jointly while preserving autoregressive computation across attention, FFN, RMSNorm, and MoE routing.
- **Privacy accounting.** We develop an RmDP formulation that composes the one-time model release with request-level token protection, yielding a smaller composed budget upper bound than a data-only baseline under the same online mechanism.
- **System and evaluation.** We instantiate the design in TWIST without modifying the provider's serving engine. Across dense and MoE models up to DeepSeek-V3.1-Terminus (671B), TWIST preserves near-plaintext utility, reduces token recovery under the evaluated provider-side attacks, and keeps online serving overhead below 10%.

2 Problem Setting

2.1 System Model

We consider tenant-dedicated BYOM serving, where a tenant provides an open-weight LLM model checkpoint and outsources inference execution to a cloud provider running standard serving infrastructure [3, 21, 29, 45]. The deployment has two phases: offline model preparation and online inference.

Offline preparation. Before deployment, the tenant uses TWIST to transform the checkpoint θ into $\widetilde{\theta}$. The tenant releases only $\widetilde{\theta}$ and retains the secret state for request processing and output recovery.

Online serving. During inference, end users interact with a tenant-controlled client. The client tokenizes requests, applies the token transformation, and sends transformed tokens to the provider. The provider executes autoregressive inference on $\widetilde{\theta}$ using an unmodified serving engine and returns transformed outputs. The client locally recovers plaintext outputs before returning them to users.

The tenant controls the client and transformation secrets, while the provider operates the serving infrastructure and performs the full forward pass. TWIST targets privacy of tenant prompts and generated outputs in this deployment model. The analysis treats the plaintext model weights as public auxiliary information.

2.2 Threat Model

We model the cloud provider as an honest-but-curious adversary: it follows the prescribed serving protocol, but it may observe all software-visible state on its machines. The tenant-side algorithms are public. We assume the tenant-controlled client remains uncompromised. We do not consider microarchitectural side channels or chosen-input access through provider-side end-user enrollment.

Offline view. During deployment, the provider receives the transformed model checkpoint $\widetilde{\theta}$. We conservatively assume that the provider also knows the plaintext checkpoint θ. This models the common BYOM case where an open-weight model is deployed, the setting evaluated in this paper. The adversary's offline recovery target is the secret token permutation π generated during deployment and embedded in the transformed model checkpoint.

Online view. During inference, the provider observes transformed token sequences exchanged with the client and may inspect runtime states generated during serving, such as activations, attention weights, KV-cache entries, router logits, and pre-softmax logits. For the analyzed single request, the states produced during inference are post-processing of the complete checkpoint and transformed input. The adversary's online recovery target is the plaintext input sequence x underlying the observed request.

Protected information and scope. TWIST aims to protect tenant prompts from provider-side recovery. The RmDP analysis in Section 5 focuses on the two formal recovery targets, namely the token permutation π and the plaintext input sequence x, under the joint offline-online mechanism. Prompt and output lengths, request timing, and cross-request history require separate analysis. We evaluate token recovery attacks empirically in Section 7.2 and Section 7.6, and discuss other potential risks in Section 8.2.

2.3 Design Goals

From this setting, we derive four design goals:

- **G1: Tenant Data Privacy.** A solution should reduce provider-side recovery of tenant inputs under the defined deployment model. We formally analyze the protection in Section 5 and evaluate it empirically against additional provider-side attacks in Section 7.
- **G2: Serving Fidelity.** Decoded responses should preserve task utility and accuracy relative to the plaintext baseline, with any degradation quantified in Section 7.
- **G3: Serving Compatibility.** The transformed model checkpoint should run on standard inference APIs and unmodified serving engines, without custom secure-computing kernels, as realized in Section 4 and Section 6.
- **G4: Low Client Computation and Serving Overhead.** Online client work should require no model forward pass, while provider-side inference should add little overhead relative to standard serving, as measured in Section 7.

2.4 Technical Preliminaries

TWIST transforms both the request-side token representation and the model parameters that process it. We briefly introduce the notation and privacy metric used below.

Token representation. Modern LLMs process discrete token IDs rather than raw text. A tokenizer, such as BPE or SentencePiece [31, 53], maps an input string into a token sequence $x \in \mathbb{Z}_n^\ell$, where n is the vocabulary size and ℓ is the sequence length. In TWIST, this token representation defines the boundary exposed to the provider during request processing and output generation. The vocabulary size n is shared by the plaintext and transformed checkpoints, whereas ℓ varies with the request. The analysis uses k as an upper bound on the number of distinct IDs in a request. Repeated occurrences contribute to ℓ but count once toward this bound. A vocabulary permutation changes token labels while preserving the sequence length and the positions of repeated tokens. In decoding and MoE routing, top-k denotes the number of candidate tokens or selected experts, respectively. These selection counts have separate meanings from the active vocabulary bound.

Model architecture. The token representation is processed by the Transformer during autoregressive inference. A Transformer consists of attention, feed-forward, normalization, and residual modules [66, 75]. Large-scale models may additionally adopt Mixture-of-Experts (MoE) architectures, where a router selects expert feed-forward modules for each token [13]. TWIST transforms these model components together with the token representation while preserving the computation structure required by standard serving engines. Section 4 describes the module-specific constructions.

Rényi-metric differential privacy and Cayley distance. In TWIST, the provider observes both the offline transformed model checkpoint and the online transformed token sequence. TWIST analyzes the privacy of these transformations using Rényi-metric differential privacy (RmDP). Unlike central differential privacy, which defines privacy over neighboring datasets, RmDP measures Rényi divergence with respect to a metric over secrets. In TWIST, the secrets are token permutations and token sequences rather than individual records. For token permutations, TWIST uses the Cayley distance on the symmetric group S_n, where one unit of distance corresponds to one vocabulary transposition. The induced

sequence distance counts the minimum number of vocabulary-ID swaps needed to consistently rename the tokens. Section 5 uses this metric to analyze the joint offline-online mechanism.

3 Design Motivation

Unlike conventional API-based inference, BYOM gives the tenant control over the model checkpoint before it is handed to the provider. This creates a design opportunity that is absent when the provider controls both the model and the serving stack. Instead of treating the deployed model as fixed and protecting only the tokens sent at inference time, the tenant can transform both sides of the inference interface: the model checkpoint before deployment and the token representation used during serving.

3.1 Why Data-Only Obfuscation Is Insufficient

One natural approach is to transform only the input before it reaches the provider. Such data-only methods include token-level substitution or perturbation [62, 73] and embedding- or representation-level noise or learned transforms [16, 52, 80]. These methods retain the original deployed model checkpoint.

In LLM inference, token representations and model parameters jointly define the computation performed by the Transformer. Although a token-level transformation changes what the provider sees, the embedding layer, internal Transformer modules, and output head remain aligned with the original coordinate system. This alignment enables embedding and representation inversion attacks that recover information about the original input from the transformed representation [15, 40]. Increasing the perturbation can make this alignment harder to exploit, but it also removes information the model needs, leading to a poor privacy–utility trade-off.

3.2 Joint Model-Token Transformation

TWIST instead transforms the model checkpoint and token interface together, so that both operate in the same transformed coordinate system. The transformed model checkpoint and transformed tokens must remain consistent throughout inference. The provider continues to execute standard autoregressive inference, but in a transformed coordinate system that is no longer aligned with the original tokens.

The challenge is to change these representations without breaking the computations used by modern LLMs. Arbitrary transformations of hidden representations would break operations such as attention, nonlinear activations, residual connections, and MoE routing. TWIST therefore combines a token permutation π, invertible weight transformations G that cancel across adjacent modules, and calibrated additive noise E to construct ϕ_Θ, preserving inference behavior while changing the provider-visible representation.

3.3 TWIST Workflow

Figure 1 shows the resulting two-phase workflow.

Offline preparation. The tenant samples (π, G, E) (a token permutation, invertible weight transformations, and additive noise), transforms the original model checkpoint, and releases only $\widetilde{\theta} = \phi_\Theta(\theta; \pi, G, E)$ to the provider. The tenant-controlled client retains the secret state, and the transformed checkpoint is deployed once on existing serving infrastructure.

Online inference. For each request, the client transforms the token sequence before sending it to the provider. The provider performs autoregressive generation using the transformed model checkpoint and returns transformed outputs. The client then recovers the plaintext outputs locally.

4 TWIST Design

TWIST jointly transforms the model checkpoint and inference-time token IDs without changing the basic computation structure used by standard Transformer inference. The construction applies transformations compatible with each module in the Transformer architecture, including the token interface, attention, FFN, normalization, residual paths, and MoE routing, while preserving inference behavior in the transformed coordinate system.

4.1 Overview

The transformation combines three components: a token permutation π, a family G of invertible weight transformations, and calibrated additive noise E. Within G, residual-side key matrices $(\widehat{P}, \widehat{Q})$ with $\widehat{P}\widehat{Q} = I$ cancel across adjacent modules. Module-local factors in G (e.g., $\widehat{U}$, $\widehat{R}$, $\widehat{H}$, $\widehat{Z}$) are chosen to preserve the computation within each module under the conditions described below. Additive noise E perturbs the released parameters and does not cancel in the forward pass. The tenant samples these components offline to construct $\widetilde{\theta} = \phi_\Theta(\theta; \pi, G, E)$, while the online client applies the token permutation and the provider performs standard autoregressive decoding on $\widetilde{\theta}$.

Sections 4.3–4.7 instantiate these transformations for the token interface, attention, FFNs, normalization and residual paths, and MoE routing.

A module transformation pairs its input and weight encodings with an output encoding ϕ_Y and decoder ψ_Y. Their identity $\psi_Y \circ \phi_Y = \mathrm{id}$ recovers the original output labels. The expected computation error compares the transformed result with the encoded original output. Compatible boundary maps let the modules compose. Appendix A defines this error and derives its propagation through residual blocks and the output head.

4.2 Offline Preparation

During offline preparation, the tenant samples a secret state (π, G, E) once and applies ϕ_Θ to obtain the transformed model checkpoint $\widetilde{\theta} = \phi_\Theta(\theta; \pi, G, E)$. The token permutation π relabels a vocabulary of size n. When it acts on embedding and head weights, we write it as the matrix $\Pi_\pi \in \mathbb{R}^{n \times n}$. The family G is the set of invertible weight transformations. It contains residual-side key pairs $(\widehat{P}, \widehat{Q})$ with $\widehat{P}\widehat{Q} = I$, plus module-local factors such as value-output couplings, rotary and scaling transforms, and block or intermediate permutations introduced with each Transformer block below (Appendix C). The tensor E is calibrated additive Gaussian noise applied before the weight transformation. The tenant retains (π, G, E) and deploys only $\widetilde{\theta}$ to the provider. Because the transformation is performed once, its cost is amortized over the deployment lifetime.

Algorithm 1 gives the offline transformation procedure. Middle-layer noise in E is Gaussian, with its standard deviation proportional

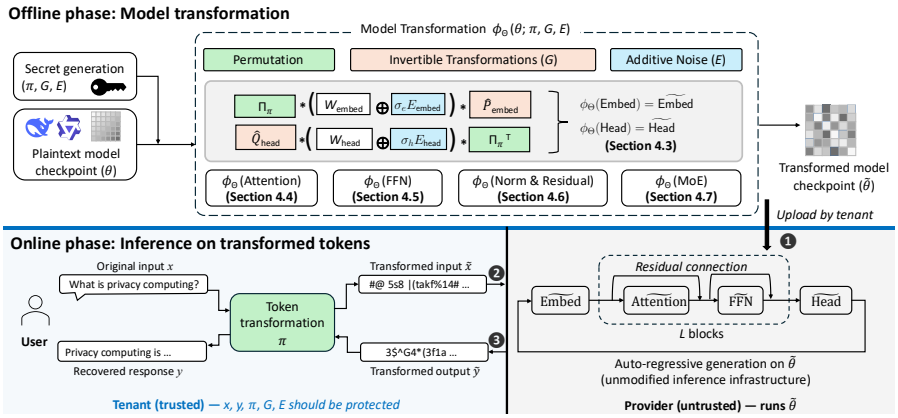


Figure 1: Conceptual workflow of TWIST. In the offline phase, the tenant samples a token permutation π (green), invertible weight transformations G (orange), and additive noise E (blue), and transforms the plaintext checkpoint θ into $\tilde{\theta} = \phi_\Theta(\theta; \pi, G, E)$. ❶ Upload $\tilde{\theta}$ to the provider. ❷ Send the transformed input $\tilde{x}$ after token transformation π. ❸ Return the transformed output $\tilde{y}$ and recover y by π^{-1}.

Algorithm 1: Offline model transformation at the client

Input: plaintext model checkpoint θ
Output: transformed model checkpoint $\tilde{\theta}$; secret state (π, G, E)

sample token permutation π invertible weight transformations G and additive noise tensors E
$(\widetilde{W}_{\text{embed}}, \widetilde{W}_{\text{head}}) \leftarrow \text{TokenTransform}(\theta; \pi, G, E)$
// Section 4.3
for *each Transformer block* **do**
  $\widetilde{W}_{\text{attn}} \leftarrow \text{AttentionTransform}(\cdot; G, E)$
  // Section 4.4
  $\widetilde{W}_{\text{ffn}} \leftarrow \text{FFNTransform}(\cdot; G, E)$ // Section 4.5
  $\widetilde{W}_{\text{norm}} \leftarrow \text{ResidualNormTransform}(\cdot; G)$
  // Section 4.6
  if *block contains MoE* **then**
    $(\widetilde{W}_{\text{router}}, \widetilde{W}_{\text{expert}}) \leftarrow \text{MoETransform}(\cdot; G, E)$
    // Section 4.7
assemble $\tilde{\theta}$ from the transformed weights
store (π, G, E) locally at the client
release $\tilde{\theta}$ to the provider
return $(\tilde{\theta}, (\pi, G, E))$

to that of the corresponding attention, FFN, or router weights. Token-interface noise multipliers remain subject to the budget of Theorem 5.3. Sampling primitives, conditioning parameters, noise parameters, and calibration are deferred to Section 6.1.

4.3 Token Interface

At the token interface, each vocabulary ID selects one embedding row and is scored by one output-head column. These weights directly link provider-visible token IDs to the continuous residual stream, and are therefore the most direct points of token-identity leakage. Let W_{embed} and W_{head} denote the plaintext embedding and head weights. To hide this correspondence, the tenant applies a secret token permutation π: online token IDs are remapped by π, and the vocabulary axes of the embedding and head are reordered by the same Π_π. Permutation alone, however, preserves row and column geometry, which a matching attack can still align against a public base model. The tenant therefore adds Gaussian noise before the algebraic transform,

$$W^\star_{\text{embed}} = W_{\text{embed}} + \sigma_e E_{\text{embed}}, \qquad W^\star_{\text{head}} = W_{\text{head}} + \sigma_h E_{\text{head}},$$

Here E_{embed} and E_{head} have independent centered Gaussian entries with standard deviations $\text{std}(W_{\text{embed}})$ and $\text{std}(W_{\text{head}})$, respectively. The parameters σ_e, σ_h are dimensionless noise multipliers.

TWIST then applies the permutation together with residual-side key matrices from G. For the token interface these are the key matrix $\widehat{P}_{\text{embed}}$ and the inverse key matrix $\widehat{Q}_{\text{head}}$, with construction given in Appendix C. The transformed token-interface weights are

$$\widetilde{W}_{\text{embed}} = \Pi_\pi \, W^\star_{\text{embed}} \, \widehat{P}_{\text{embed}}, \qquad \widetilde{W}_{\text{head}} = \widehat{Q}_{\text{head}} \, W^\star_{\text{head}} \, \Pi_\pi^\top.$$

Here Π_π reorders embedding rows, while $\Pi_\pi^\top = \Pi_\pi^{-1}$ reorders the head's vocabulary axis into the same provider-visible labeling.

The same token permutation π is applied to input and output token IDs, so the client remaps plaintext IDs by π and recovers outputs by π^{-1}. The embedding and output head use the same vocabulary permutation, keeping token IDs consistent throughout autoregressive generation. Adjacent layers are left- and right-multiplied by a key matrix and its inverse from G, so these factors cancel in the forward pass ($\widehat{P}\widehat{Q} = I$). The transformed weight matrices are no longer expressed directly in the original coordinates.

4.4 Attention

Attention [66] has two parallel branches over a shared residual input: the query-key branch forms scores by inner products, and the value-output branch aggregates content. The two branches therefore require different transforms (Theorem A.1, Theorem A.2).

For each attention group, TWIST applies an intra-head transformation together with an inter-head permutation.

For query/output head i and its paired key/value head $\eta(i)$, let $W_q^{(i)}$, $W_k^{(\eta(i))}$, $W_v^{(\eta(i))}$, and $W_o^{(i)}$ denote the plaintext query, key, value, and output weights. Residual-side key matrices $\widehat{Q}_q, \widehat{Q}_k, \widehat{Q}_v, \widehat{P}_o$ provide the two-sided canceling transform across the residual stream. A value-output coupling $\widehat{U}_{vo}$ (with inverse on the output path) preserves value aggregation. Rotary-compatible $\widehat{R}_{qk}$ and scaling $\widehat{H}_{qk}$ (with matching inverse on the key path) preserve query-key scores under RoPE. A block permutation $\widehat{Z}_{\text{block}}$ is applied with the same orientation on the query and key paths, preserving scores before positional rotation. With RoPE, exact preservation additionally requires compatibility with the rotary frequencies. Mixing different rotary frequencies within a window introduces approximation error. The transformed weights are constructed as

$$\begin{aligned}
\widetilde{W}_q^{(i)} &= \widehat{Q}_q \,(W_q^{(i)} + E_q^{(i)})\, \widehat{R}_{qk}\, \widehat{H}_{qk}\, \widehat{Z}_{\text{block}},\\
\widetilde{W}_k^{(\eta(i))} &= \widehat{Q}_k \,(W_k^{(\eta(i))} + E_k^{(\eta(i))})\, \widehat{R}_{qk}\, \widehat{H}_{qk}^{-1}\, \widehat{Z}_{\text{block}},\\
\widetilde{W}_v^{(\eta(i))} &= \widehat{Q}_v \,(W_v^{(\eta(i))} + E_v^{(\eta(i))})\, \widehat{U}_{vo},\\
\widetilde{W}_o^{(i)} &= \widehat{U}_{vo}^{-1} \,(W_o^{(i)} + E_o^{(i)})\, \widehat{P}_o.
\end{aligned}$$

Here $E_a^{(i)}$ denotes the additional offline Gaussian noise on each attention-path matrix, sampled by the standard-deviation-scaled rule above. The concrete construction of these matrices is given in Appendix C.

TWIST additionally permutes attention heads and head groups. For grouped-query attention, one permutation reorders key/value heads together with their associated query/output groups. A second permutation reorders query/output heads within each group. Each query therefore retains its corresponding key/value head, and the output weights follow the same head ordering. The construction specializes to multi-head attention when each query has its own key/value head and to multi-query attention when all queries share one. Within a head, the windowed permutation acts on pairs of rotary coordinates. Its maximum window size is β, which limits how far these blocks can be rearranged. This permutation removes the direct cross-head correspondence that a matching attack could otherwise exploit. The construction covers multi-head, multi-query, and grouped-query attention [1, 54]. For multi-head latent attention (MLA) [13], TWIST uses path-specific invertible transformations to preserve its latent low-rank structure and decoupled positional pathway.

Proposition 4.1 (Attention preservation). *Let x denote a residual-state input, $\widetilde{x} = x\widehat{P}$, $q = xW_q$, and $k = xW_k$. Assume zero additive weight noise and $\widehat{P}\widehat{Q}_q = \widehat{P}\widehat{Q}_k = \widehat{P}\widehat{Q}_v = I_d$. Let $\widehat{R}_{qk}$ contain two-dimensional rotations, let $\widehat{H}_{qk}$ contain nonzero scalar scales on those pairs, and let $\widehat{U}_{vo}$ be invertible. Before positional rotation,*

$$(\widetilde{x}\widetilde{W}_q)(\widetilde{x}\widetilde{W}_k)^\top = qk^\top, \qquad (\widetilde{x}\widetilde{W}_v)\widetilde{W}_o = (xW_v)W_o\widehat{P}_o.$$

The score identity also holds after RoPE when the block permutation commutes with each relative-position rotation used by the request.

The proof and the score-error bound for frequency mixing are given in Appendix C. The bound depends on frequency differences and the separation between token positions. With additive noise, the error also includes perturbations to the projection weights.

4.5 Feed-Forward Path

The FFN path places an elementwise nonlinearity between linear maps, so an arbitrary change of intermediate basis would not commute with the activation. TWIST therefore restricts the intermediate-space transform to a permutation-plus-diagonal-scaling that preserves the gate-up-down pattern.

Let $W_{\text{gate}}, W_{\text{up}}, W_{\text{down}}$ denote the FFN weights. As in attention, residual-side key matrices $\widehat{Q}_{\text{gate}}, \widehat{Q}_{\text{up}}, \widehat{P}_{\text{down}}$ cancel with the residual stream, while module-local $\widehat{Z}_{\text{ffn}}$ (permutation) and $\widehat{H}_{\text{ffn}}$ (positive diagonal) act only in the intermediate space and are absorbed on the down path. TWIST constructs

$$\begin{aligned}
\widetilde{W}_{\text{gate}} &= \widehat{Q}_{\text{gate}}\,(W_{\text{gate}} + E_{\text{gate}})\,\widehat{Z}_{\text{ffn}},\\
\widetilde{W}_{\text{up}} &= \widehat{Q}_{\text{up}}\,(W_{\text{up}} + E_{\text{up}})\,\widehat{H}_{\text{ffn}}\,\widehat{Z}_{\text{ffn}},\\
\widetilde{W}_{\text{down}} &= \widehat{Z}_{\text{ffn}}^{-1}\,\widehat{H}_{\text{ffn}}^{-1}\,(W_{\text{down}} + E_{\text{down}})\,\widehat{P}_{\text{down}},
\end{aligned}$$

where E_a denotes the additional offline Gaussian noise for each $a \in \{\text{up}, \text{gate}, \text{down}\}$, sampled by the same standard-deviation-scaled rule. The gate and up branches share $\widehat{Z}_{\text{ffn}}$, which keeps their elementwise product aligned. The down projection absorbs the inverse intermediate factors and produces the transformed residual representation. Before additive noise, this transform preserves the FFN computation under the stated compatibility conditions. The placement of the diagonal scale is essential: it multiplies the linear up branch, while the gate activation receives only permuted coordinates. After the two branches are multiplied elementwise, the down projection removes that scale. This order preserves the nonlinear gate without requiring the activation to commute with scaling.

Proposition 4.2 (FFN activation preservation). *Let*

$$y = \big(\text{SiLU}(xW_{\text{gate}}) \odot (xW_{\text{up}})\big)W_{\text{down}}, \qquad \widetilde{x} = x\,\widehat{P}.$$

If $\widehat{P}\widehat{Q}_{\text{gate}} = \widehat{P}\widehat{Q}_{\text{up}} = I$, $\widehat{Z}_{\text{ffn}}$ is a permutation, $\widehat{H}_{\text{ffn}}$ sits only on the up branch and is absorbed into the left factor of $\widetilde{W}_{down}$, and the activation is applied element-wise after that permutation, then (before noise)

$$\widetilde{y} = \big(\text{SiLU}(\widetilde{x}\,\widetilde{W}_{gate}) \odot (\widetilde{x}\,\widetilde{W}_{up})\big)\widetilde{W}_{down} = y\,\widehat{P}_{\text{down}}.$$

The same identity holds for GELU and ReLU under permutation-plus-diagonal intermediate maps. It fails under arbitrary orthogonal maps, because element-wise nonlinearities do not commute with rotations.

4.6 Normalization and Residual Compatibility

For RMSNorm, the learned channel gain can be fused into an adjacent linear projection before applying the key transformation. The remaining scalar normalization depends on the residual width and stabilizing constant. For $d' = d + 2h$, write

$$r_d(x) = \sqrt{\|x\|_2^2/d + \epsilon_d}.$$

The exact scalar correction at activation x is $r_{d'}(x\widehat{P})/r_d(x)$. A scalar κ estimated from calibration approximates this ratio. With zero stabilizers, the ratio includes the dimension factor $\sqrt{d/d'}$ as well as $\|x\widehat{P}\|_2/\|x\|_2$. Appendix C derives the correction and its relation to

normalization error. The learned gain and the norm correction thus serve different purposes. The gain retains the original channel-wise weights in the adjacent projection, while the scalar correction compensates for the change in activation norm caused by the key transformation. Using a calibrated scalar leaves an activation-dependent normalization error.

Residual addition requires the same realized output encoding on both branches: $(x+u)\widehat{P} = x\widehat{P} + u\widehat{P}$. Subsequent projections use compatible right inverses to recover the shared residual coordinate.

4.7 Mixture-of-Experts

For MoE models, the router determines which experts are selected, and its logits may expose token-dependent routing structure if left unchanged. Expert identities may correlate with semantic specialization, and the top-k routing decision can leak information about token content. TWIST protects the router by first normalizing each expert's weight vector to obtain W'_{router} and then applying a residual-side key matrix together with an expert permutation $\widehat{Z}_{\text{router}} \in \mathbb{R}^{n_{\text{exp}} \times n_{\text{exp}}}$ on the set of n_{exp} experts:

$$\widetilde{W}_{\text{router}} = \widehat{Q}_{\text{router}} \, (W'_{\text{router}} + E_{\text{router}}) \, \widehat{Z}_{\text{router}}.$$

Matching left factors transform each expert's weights so that the permuted router selects the correct experts.

For a given router logit vector, the expert permutation preserves the selected set up to relabeling, with consistent tie-breaking. The expert functions and their aggregation weights are reordered by that same permutation. Expert-local hidden transformations follow the FFN compatibility rules of Section 4.5.

Let $r^\star = x(W'_{\text{router}} + E_{\text{router}})$ be the logits of the perturbed router. When $\widehat{P}\widehat{Q}_{\text{router}} = I$, $\widetilde{r} = r^\star \widehat{Z}_{\text{router}}$, so

$$\text{TopK}(\widetilde{r}) = \widehat{Z}^{-1}_{\text{router}}\big(\text{TopK}(r^\star)\big).$$

Agreement with the original selected set additionally depends on the total logit error $\delta r = x(W'_{\text{router}} + E_{\text{router}} - W_{\text{router}})$. It holds when the gap between the kth and $(k+1)$th original logits exceeds twice the maximum absolute logit error. Appendix C gives the corresponding permutation and margin arguments.

4.8 End-to-End Construction

Applying the transformations above yields the deployed checkpoint $\widetilde{\theta} = \phi_\Theta(\theta; \pi, G, E)$. The end-to-end analysis tracks where each error enters the model. An embedding error passes through all decoder blocks, whereas an error introduced in a later block passes through only the remaining blocks and the output head. Each contribution is therefore weighted by the sensitivity of the downstream modules when bounding the final logit error. The composition rules and the end-to-end error bound appear in Appendix A. For embedding error e_0, block errors e_i, and head error e_{h}, the logit error obeys

$$e_{\text{logit}} \le M_{\text{h}} \left(e_0 \prod_{j=1}^{L} M_j + \sum_{i=1}^{L} e_i \prod_{j=i+1}^{L} M_j \right) + e_{\text{h}},$$

where M_j, M_{h} bound the corresponding transformed modules' sensitivity on the stated input domain. For greedy decoding, a maximum logit error below half the leading-token margin preserves the token choice. Appendix A derives the residual and autoregressive bounds.

4.9 Online Inference Protocol

Algorithm 2 shows the online exchange between the tenant-controlled client and the provider. The client detokenizes the mapped IDs into an obfuscated prompt, which the provider tokenizes for inference. Serialization and tokenization must preserve the mapped IDs in both directions of the exchange. Algorithm 2 abstracts these serialization and re-tokenization steps as sending the transformed token IDs. The client reverses the secret vocabulary permutation on the returned tokens. The model processes the perturbed input produced by M_{exp}. Section 7 evaluates the resulting utility loss.

Algorithm 2: Client-provider online inference

Input: prompt string s; token permutation π; budget ε_{on}; tokenizer $\mathcal{T}$; deployed $\widetilde{\theta}$
Output: response string

`/* Client */`
$x \leftarrow \mathcal{T}(s)$
sample $\widehat{x} \sim M_{\text{exp}}(x)$ at budget ε_{on}
$\widetilde{x} \leftarrow \pi(\widehat{x})$
send $\widetilde{x}$ to the provider
`/* Provider */`
run standard autoregressive decoding of $\widetilde{\theta}$ on $\widetilde{x}$
return provider-visible output token IDs $\widetilde{y}$
`/* Client */`
$y \leftarrow \pi^{-1}(\widetilde{y})$
return $\mathcal{T}^{-1}(y)$

4.10 Complexity and Serving Compatibility

The offline transformation processes each weight once while retaining the model vocabulary and standard Transformer operator structure. The hidden width expands from d to $d+2h$ under the key-matrix construction of Appendix C. Per-request client work is one M_{exp} sample on the request sequence, plus applying π (and π^{-1}) to token IDs. The provider runs standard autoregressive decoding with KV-cache layout, paged attention, continuous batching, and streaming output unchanged at the engine level [33, 78]. The main structural difference is the widened hidden dimension. Section 6 and Section 7 report implementation details and measured overhead.

5 Rényi-Metric Privacy Analysis

Section 5 analyzes the offline checkpoint release and online token mechanism separately before composing their privacy bounds.

5.1 From DP to Rényi-Metric DP

Definition 5.1 (Rényi-metric differential privacy)**.** Let (S, d) be a metric space of secrets and let $M : S \to \mathcal{Y}$ be a randomized mechanism. For $\alpha > 1$ and $\varepsilon \ge 0$, M satisfies (α, ε, d)-*Rényi-metric DP* if

$$D_\alpha\big(M(s) \,\|\, M(s')\big) \;\le\; \varepsilon \, d(s, s') \quad \text{for all } s, s' \in S,$$

where D_α is the Rényi divergence of order α. The variant $(\alpha, \varepsilon, d^2)$-Rényi-metric DP requires the same bound with $d(s, s')^2$ in place of $d(s, s')$.

Concentrated geo-privacy [36] requires the squared-distance bound for every $\alpha > 1$ with $\varepsilon(\alpha) = \alpha\rho$ for a common ρ. Here we also use fixed-order bounds with linear distance, as specified for each mechanism.

Both forms are closed under post-processing of the analyzed mechanism's output.

Two primitive randomized mechanisms are combined into $M^{\partial}_{\text{full}}$: the Gaussian mechanism and an exponential mechanism.

Gaussian mechanism. As in Rényi DP, we have the following theorem about the Gaussian mechanism for RmDP.

Theorem 5.1 (Gaussian Mechanism for RmDP). *Let $(\mathcal{X}, d)$ be a metric space, and let $f : \mathcal{X} \to \mathbb{R}^d$ be a query function satisfying*

$$L_2(f) := \sup_{x \neq x' \in \mathcal{X}} \frac{\|\Sigma^{-1/2}(f(x) - f(x'))\|_2}{d(x, x')} < \infty,$$

where Σ is a symmetric positive definite matrix.

The Gaussian mechanism is defined by adding Gaussian noise $\mathcal{N}(0, \sigma^2\Sigma)$ to the query output:

$$\mathcal{M}(x) = f(x) + \mathcal{N}(0, \sigma^2\Sigma).$$

This mechanism satisfies (α, ϵ, d^2)-RmDP with

$$\epsilon = \frac{\alpha L_2(f)^2}{2\sigma^2}.$$

Exponential token mechanism. Let X_ℓ be the set of sequences obtained by consistently renaming the token IDs of a fixed length-ℓ sequence. For $x \in X_\ell$ and an online budget parameter $\varepsilon_{\text{on}} > 0$, define

$$\Pr\big[M_{\text{exp}}(x) = z\big] \;\propto\; \exp\big(-\varepsilon_{\text{on}}\, d(x, z)\big), \qquad z \in X_\ell.$$

Theorem 5.2 (Rényi-metric bound for M_{EXP}). *For every $\alpha > 1$, M_{exp} satisfies $(\alpha, \varepsilon_{on}, d)$-RmDP on every such set X_ℓ.*

The client runs M_{exp} online. Theorem 5.2 establishes this privacy guarantee for the Cayley distance d (Appendix B).

5.2 Recovery Targets, Metrics, and Provider Mechanism

We analyze two provider-side recovery targets: (1) the *offline* attacker inspects the deployed transformed model checkpoint and tries to recover the token permutation $\pi \in S_n$, and (2) the *offline-online* attacker additionally observes transformed input tokens and tries to recover the plaintext token sequence x. The key matrices enter the analysis as internal randomness in the offline release.

Cayley distance. We use the Cayley distance introduced in Section 2.4. For a fixed request length ℓ, two sequences are comparable when one can be obtained from the other by a global renaming of vocabulary IDs. Equivalently, the same positions must contain repeated tokens in both sequences. The distance is the minimum number of transpositions in S_n needed to realize such a renaming. Set $d(x, x') = \infty$ when the sequences have different lengths or equality patterns.

Provider view. Write $M_{\text{off}}(\pi)$ for the complete released checkpoint, including its normalization and configuration fields. For one plaintext request x, we analyze

$$M^{\partial}_{\text{full}}(x) = \big(M_{\text{off}}(\pi), \widetilde{x}\big), \qquad \widetilde{x} = \pi(M_{\text{exp}}(x)).$$

The analysis begins with the checkpoint and an empty request history. The provider's outputs and runtime states for this request follow from this pair through the serving computation and independent random coins. They therefore inherit its bound by post-processing. The offline theorem compares secrets π. The joint theorem averages over π to compare plaintext requests x.

5.3 Child Mechanism: Offline Model Checkpoint Leakage

We bound token-permutation leakage from the complete checkpoint released by the offline mechanism $M_{\text{off}}(\pi)$.

Theorem 5.3 (Offline budget for the deployed model checkpoint). *Let $W_{embed} \in \mathbb{R}^{n\times d}$ and $W^{\top}_{head} \in \mathbb{R}^{n\times d}$ be the plaintext embedding and transposed head matrices. Let $s_e = \sigma_e \,\text{std}(W_{embed})$ and $s_h = \sigma_h \,\text{std}(W_{head})$ be the absolute Gaussian standard deviations. The offline construction of Section 4 uses independent centered Gaussian noise. Its noise arrays and remaining offline randomness are mutually independent and independent of π. All noise standard deviations below are positive. For a fixed plaintext checkpoint and public release parameters, the complete checkpoint is a common function of the noisy token interfaces and the remaining offline randomness. Define the normalized complex token-interface matrix*

$$A_{\partial} \;=\; \frac{1}{s_e} W_{embed} \;+\; i\,\frac{1}{s_h} W^{\top}_{head} \;\in\; \mathbb{C}^{n\times d}.$$

Let $\lambda_1 \geq \lambda_2$ be the top two singular values of A_∂ (spectral quantities, not the key-matrix conditioning parameter λ of Appendix C). For the intermediate transformer layers, let

$$\mathcal{J} \;=\; \{q, k, v, o, \text{gate}, \text{up}, \text{down}, \text{router}\},$$

and let $W_{t,j}$ be the plaintext matrix for each component j present at layer t, with absolute Gaussian standard deviation $s_{t,j} = \rho\,\text{std}(W_{t,j})$. The sum includes the components present at each layer. Define the middle-layer spectral summary

$$\varepsilon_{mid} \;=\; \alpha \sum_{t=1}^{L} \sum_{j\in\mathcal{J}} \frac{\lambda_1(W_{t,j})^2 + \lambda_2(W_{t,j})^2}{2s^2_{t,j}}.$$

Define

$$\mu_{1,2} \;=\; \lambda_1^2 + \lambda_2^2 - \frac{4\lambda_1^2\lambda_2^2}{\lambda_1^2 + \lambda_2^2 + 4\varepsilon_{mid}/\alpha}.$$

Then, for every $\alpha > 1$, M_{off} satisfies $(\alpha, \varepsilon_{off}, d^2)$-RmDP on the token permutation π, with

$$\varepsilon_{off} \;=\; \frac{\alpha\varpi^2}{2} \max\{\mu_{1,2}, \lambda_1^2\}.$$

Here ϖ denotes the circle constant, written distinctly from the token permutation π.

The proof bounds the two noisy interfaces, then applies post-processing to obtain the complete release. The proof of Theorem 5.3 is provided in Section B.6.

5.4 Offline-Online Joint Mechanism

Combining the offline model release with the online token mechanism gives the joint mechanism for each request

$$M^{\partial}_{\text{full}}(x) \;=\; \big(M_{\text{off}}(\pi),\; \pi(M_{\text{exp}}(x))\big),$$

where M_{exp} is the exponential token mechanism of Section 5.1. Here π is uniform on S_n and independent of the request. The offline and online draws are independent conditional on π. Write $\widetilde{x}_{1:\ell} = \pi(M_{\text{exp}}(x))$ for the provider-visible token sequence.

The actual returned token stream is produced by standard decoding from the full transformed model checkpoint. The provider's serving computation is post-processing of the checkpoint and transformed request. The offline checkpoint budget contributes to the joint privacy bound, while the exponential mechanism controls the per-request online part. Theorem 5.4 gives the resulting composition bound.

Theorem 5.4 (Offline-Online Joint Mechanism). *Assume the one-time offline release M_{off} satisfies $(\alpha > 1, \varepsilon_{off}, d^2)$-RmDP on the token permutation π from Theorem 5.3. Assume M_{exp} satisfies $(\alpha > 1, \varepsilon_{on}, d)$-RmDP on plaintext token sequences x from Theorem 5.2. Then the request-level parent mechanism M^{∂}_{full} satisfies $(\alpha, \varepsilon_{full}, d)$-RmDP on $\mathbb{Z}_n^{\ell}(k) := \{x \in \mathbb{Z}_n^{\ell} : |\text{set}(x)| \le k\}$, the set of length-$\ell$ token sequences over a vocabulary of size n whose number of distinct tokens is at most the active vocabulary size k, with*

$$\varepsilon_{full} = \begin{cases} \varepsilon_{on} - \dfrac{\varepsilon_{on}^2}{4(k-1)\varepsilon_{off}} & \text{if } \varepsilon_{on} \le 2(k-1)\varepsilon_{off}, \\ (k-1)\,\varepsilon_{off} & \text{otherwise}, \end{cases}.$$

The budget $\varepsilon_{\text{full}}$ is non-negative and non-decreasing in both ε_{on} and ε_{off} on the two regimes, and the two branches meet at the boundary $\varepsilon_{\text{on}} = 2(k-1)\varepsilon_{\text{off}}$.

Theorem 5.4 shows how the joint bound changes with the active vocabulary size k. When the closed-form offline parameter ε_{off} is small, the second branch scales linearly with k, $\varepsilon_{\text{full}} = (k-1)\varepsilon_{\text{off}}$, until the switch condition $\varepsilon_{\text{on}} \le 2(k-1)\varepsilon_{\text{off}}$ is met. After the switch point, $\varepsilon_{\text{full}}$ approaches ε_{on} from below as k increases. Remark 5.1 interprets the three parameters and the comparison with data-only obfuscation.

The complete proof appears in Appendix G.

Remark 5.1 (Interpretation and scope of the RmDP parameters). The three RmDP parameters correspond to different parts of the joint mechanism. They measure Rényi divergence growth under the metric defined on the secret space, which captures token permutation or sequence distance. Specifically, ε_{on} corresponds to the request-level exponential mechanism, ε_{off} bounds the one-time transformed model checkpoint release carrying uncertainty about π, and $\varepsilon_{\text{full}}$ captures their composition in Theorem 5.4. The resulting bound depends on the active vocabulary size through the $(k-1)$ factor. The formal analysis in Theorem 5.4 provides a structural and comparative guarantee. Its numerical bound remains loose at the default evaluation setting, where $\varepsilon_{\text{full}} \approx 20$. The comparison holds the online mechanism constant and examines how the offline model transformation contributes to the joint bound. Compared with a data-only mechanism that uses the same online protection, which has $\varepsilon_{\text{full}} = \varepsilon_{\text{on}}$, TWIST achieves $\varepsilon_{\text{full}} < \varepsilon_{\text{on}}$ by transforming the model offline. The closed-form offline parameter ε_{off} from Theorem 5.3 may remain loose under practical noise scales. Developing tighter accounting for the offline transformation remains future work. The practical privacy claim of this paper rests primarily on the empirical attack evaluation of Section 7.

6 Implementation

Our implementation consists of an offline model transformation tool and a client-side token processing program. The transformation tool produces a model checkpoint for deployment with the vLLM inference engine [33] and a corresponding client-side tokenizer. The following subsections describe offline transformation, online serving integration, and request-level sampling.

6.1 Offline Transformation

The offline implementation follows Algorithm 1. The implementation loads the checkpoint and samples the secret state: a vocabulary permutation π, weight transformations in G via the expanded near-orthogonal key-matrix construction of Appendix C, and calibrated additive Gaussian noise E. Conditioning λ and expansion h are chosen for fp16/bf16 serving. It then applies the per-component transformations of Section 4.3–Section 4.7 and validates numerical conditioning and commutation on a calibration set of 512 held-out prompts. The implementation writes the transformed model checkpoint. For each attention, FFN, and router matrix W, the implementation samples additional offline noise as $E = \rho\ \text{std}(W)\,\Xi$ with $\Xi \sim \mathcal{N}(0, I)$ and default $\rho = 0.05$. Token-interface multipliers σ_e and σ_h and the remaining architectural parameters are selected against the same held-out set under the budget of Theorem 5.3. The token-interface noise is scaled in the coordinates of the plaintext weights. Each Gaussian array has the standard deviation of its corresponding weight matrix, and the multiplier specifies the noise relative to that scale. Thus $\sigma_e = 1$ gives embedding noise with standard deviation $\text{std}(W_{\text{embed}})$, while $\sigma_h = 0.2$ gives head noise with standard deviation $0.2\,\text{std}(W_{\text{head}})$. The absolute standard deviations s_e and s_h enter the offline analysis. The tenant adds this noise before applying the vocabulary permutation and key matrices, as specified in Section 4.3. The calibration set is also used to validate utility and numerical stability under this default noise parameter. The client stores the inverse key matrices in a separate encrypted artifact, which remains within the tenant's control. The transformation runs once on commodity hardware: 3.43 min on R1-Distill-14B and 482.38 min (about 8 hours) on DeepSeek-V3.1-Terminus, amortized over deployment lifetime (Table 5).

6.2 Online Serving Integration

The client uses the secret token mapping generated during offline transformation to obfuscate input prompts. This mapping uses the same permutation π applied to the embedding and model head. The client first tokenizes the prompt locally. With online perturbation enabled, it samples from the exponential mechanism once per request before applying the secret token mapping.

The client then detokenizes the mapped tokens to construct an obfuscated prompt and sends it to the provider. The provider tokenizes this prompt to obtain the input IDs for the transformed model. Input and output serialization must preserve the mapped IDs in Algorithm 2. The model produces an obfuscated response, which the client decodes through π^{-1} to obtain plaintext. Streaming outputs are decoded token-by-token.

The provider requires no custom kernels or runtime changes: transformed model checkpoints serve through the standard engine

Table 1: Default hyperparameters used in the evaluation.

Symbol	Value	Meaning
Privacy		
α	2	Rényi order
ε_{on}	20	per-request sampler budget
$\varepsilon_{\text{full}}$	≤ 20	joint bound from ε_{on}
k	100	active vocabulary bound
ε_{off}	3.47×10^{9}	reported offline value
σ_e	1.0	embedding noise multiplier
σ_h	0.2	head noise multiplier
ρ	0.05	middle-layer noise multiplier
Offline transformation		
λ	0.3	key-matrix parameter
h	128	intermediate expansion size
β, γ	$8, 10^3$	attention block permutation
Inference		
temperature	0.65	decoding temperature
top-k, top-p	20, 0.95	decoding cutoffs
max-seq-len	16384	context window
dtype	bfloat16	serving precision

Note: k above is the analysis bound in Theorem 5.4, not the decoding top-k. ε_{off} is for Qwen2.5-14B-Instruct. Other models are in Table 7.

because the deployed model checkpoint keeps the same vocabulary and transformer operator structure, with hidden dimension expanded from d to $d+2h$. KV cache, paged attention [33], continuous batching, and streaming all work unchanged on this expanded shape [78].

6.3 Online RmDP Sampler

The client applies an exponential mechanism for token perturbation before applying the secret token mapping. As defined in Section 5.1, the mechanism samples over token sequences obtained by applying vocabulary permutations to the input sequence. Each permutation acts on token IDs throughout the sequence, so all occurrences of the same token ID are mapped consistently.

The finite-ε_{on} configurations sample once per request, before the model forward pass. The ∞ configuration in Figure 3 applies the secret token mapping directly to the original request. Changing ε_{on} at deployment time requires no model retraining or redeployment.

7 Evaluation

7.1 Evaluation Setting

Models and datasets. We use Qwen2.5-14B-Instruct [51] for baseline comparisons, ablations, and privacy-utility analysis. We additionally evaluate five models (Llama3-8B [22], Qwen3-14B/32B [72], and R1-Distill-14B/32B [12]) in the 8B–32B range. For MoE architectures, we include Qwen3-MoE-30B-A3B and DeepSeek-V3.1-Terminus (671B) [11]. Utility benchmarks cover knowledge (MMLU [26], C-Eval [28]), coding (HumanEval [7]), reasoning (PIQA [4]), instruction following (IFEval [79]), and sentiment (SST-2 [56]). We use PUPA [35] to evaluate the protection of personally identifiable information (PII) against single-request recovery attacks. We use CCI3 [69], Huatuo26M-Lite [70], and MedDialog [8] for frequency analysis.

Baselines and metrics. `SANTEXT` [73] and `RANTEXT` [62] perform discrete substitution or perturbation on raw input tokens. Our `DP-Forward` [16] adaptation adds calibrated Gaussian noise to the hidden states after the first decoder layer. Stained Glass (SGT) [52] replaces analytical noise with a learned transformation to balance utility and privacy. The comparison uses Qwen2.5-14B-Instruct as the shared base model.

The attacks examine three sources of provider-visible information. Vocabulary-Matching Attack (VMA) [61] and Invariant Attack (IA) [40] use relationships between known plaintext weights and transformed weights to infer the hidden token mapping. VMA combines matching results from multiple decoder layers by voting, while IA constructs signatures that are invariant to parts of the weight transformation. Internal State Attack (ISA) [15], Inversion Model Attack (IMA) [32], and nearest-neighbor (NN) attack seek to recover inputs from representations observed during inference through optimization, a trained inversion model, and nearest-neighbor matching, respectively. TFMA and SDA use token-frequency information accumulated under the same permutation to recover tokens or text. Collectively, these attacks cover three provider-visible surfaces: transformed weights, runtime states, and repeated token observations.

TTRSR measures the proportion of original tokens successfully recovered by the attacker. Recovered-Text Cosine Similarity (CosSim) calculates the cosine similarity between the embedding of the recovered text and the original plaintext. CosSim captures partial semantic recovery even when exact token matching fails. We measure PIIRSR as the personally identifiable information recovery rate on the same inversion outputs. Section 7.6 evaluates token-frequency leakage with Token Frequency Matching Attack (TFMA) and Substitution Deciphering Attack (SDA) [2]. TFMA is reported as TTRSR at top-10 and top-100, and SDA as a BLEU-4 score. For all recovery metrics, lower values indicate stronger privacy.

Hardware and software. The client uses two Intel Xeon Platinum 8457C processors (96 cores). The provider runs vLLM 0.9.1 [33] with CUDA 12.4 on a GPU cluster. Further evaluation details appear in Appendix I.

Hyperparameter configuration. Table 1 lists the default `TWIST` configuration used throughout the evaluation unless stated otherwise.

7.2 Privacy-Utility Performance

Table 2 compares TWIST with four data-only baselines on Qwen2.5-14B-Instruct. `SANTEXT` and `RANTEXT` expose more than half of input tokens by direct observation, at TTRSR 62.90% and 56.92% respectively. `RANTEXT` also drops PIQA by 59 points and `SANTEXT` drops MMLU by 31. `DP-Forward` retains 93.60% accuracy on SST-2 but reaches 77.08% TTRSR under IMA. Thus, preserving task accuracy alone does not imply input privacy. `SGT` uses a learned 1.6B-parameter transformation rather than analytical noise, but its MMLU and PIQA scores fall to 30.30 and 26.06. IMA recovers 98.58% of `SGT`'s input tokens at CosSim 0.99.

`TWIST` stays within 2.40 percentage points of `Plaintext` on all four benchmarks, while keeping TTRSR at or below 13.51% across

Table 2: Utility and privacy on Qwen2.5-14B-Instruct.

	Utility (Acc. %) ↑				Privacy ↓		
Method	SST2	MMLU	PIQA	IFEval	Attack	TTRSR (%)	CosSim
`Plaintext`	97.25	81.95	77.45	77.09	—	—	—
`SANTEXT`	81.31	51.07	49.72	57.45	Direct obs.	62.90	0.93
`RANTEXT`	87.50	37.77	18.30	57.18	Direct obs.	56.92	0.88
`DP-Forward`[a]	93.60	—	—	—	NN	37.82	0.51
					IMA	77.08	0.84
`SGT`[b]	92.43	30.30	26.06	34.66	NN	11.12	0.69
					IMA	98.58	0.99
TWIST	**97.13**	**80.61**	**75.23**	**79.49**	VMA	13.51	0.31
					NN	0.00	0.42
					IMA	0.00	0.36
					IA	5.95	0.40
					ISA	0.00	0.20

Note: Privacy metrics cover input tokens only. The baselines do not protect generated content.

[a] Utility evaluation of our `DP-Forward` adaptation covers SST-2 only.

[b] `SGT` source is unreleased. We re-implement it from [52].

Table 3: Ablation on Qwen2.5-14B-Instruct. Each row adds one transformation component and reports ISA TTRSR from attention scores and hidden states. Lower indicates less token recovery.

Configuration	AttnScore	HiddenState
Noise only	87.14	40.00
+ Two-sided key matrices	87.14	0.82
+ Head-and-block permutations	0.00	0.00

the five attacks. Only VMA and IA recover a nonzero fraction of exact tokens, at 13.51% and 5.95% TTRSR, respectively.

7.3 Component Ablation

Table 3 measures the incremental contribution of transformation components to token recovery from attention scores and hidden states. Starting from noise alone, we add the two-sided key matrices and then the head-and-block permutations. We run ISA separately on attention scores and decoder-layer hidden states [15].

The two-sided key matrices change the hidden-state coordinates but cancel from the query-key score computation (Proposition 4.1). This explains why hidden-state recovery falls from 40.00% to 0.82% while attention-score recovery remains at 87.14%. Adding head-and-block permutations then reduces TTRSR to 0.00% for both attack inputs, consistent with removing the head/block alignment available to ISA.

7.4 Parameter Sensitivity

We vary the token-interface noise and online privacy parameter to characterize the privacy-utility trade-off.

Increasing embedding noise σ_e from 0.5 to 1.5 reduces VMA TTRSR from 31.12% to 6.74%, at a cost of 5.39 C-Eval points (Figure 2). At head noise $\sigma_h = 0.3$, TTRSR falls below 10% while C-Eval

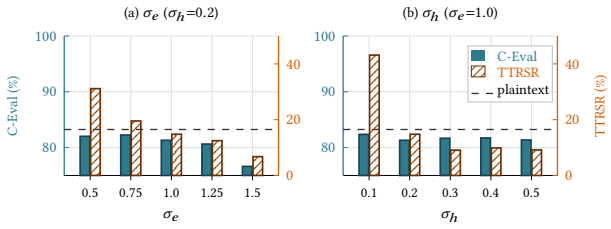


Figure 2: VMA token recovery (TTRSR) and C-Eval accuracy as token-interface noise varies on Qwen2.5-14B-Instruct. (a) Varying σ_e with fixed σ_h=0.2. (b) Varying σ_h with fixed σ_e=1.0. The dashed line indicates plaintext C-Eval (83.24).

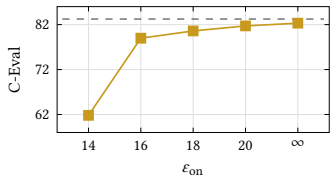


Figure 3: C-Eval accuracy as the online privacy parameter ε_{on} varies on Qwen2.5-14B-Instruct. At the finite settings, $\varepsilon_{full} \approx \varepsilon_{on}$ at the reported precision, with 20 as the default. The ∞ setting omits online perturbation.

remains above 81. Noise in these weights disrupts the relationships used by VMA. With little noise, selected weight combinations remain close to row and column permutations of their plaintext counterparts, allowing sorting and matching to recover token correspondences.

Holding $(\sigma_e, \sigma_h, \rho) = (1.0, 0.2, 0.05)$ fixed, we vary ε_{on} (Figure 3). Around the default $\varepsilon_{on} = 20$, C-Eval plateaus near 81.7, 1.53 points below plaintext.

At these noise scales, the closed-form offline parameter remains large: $\varepsilon_{off} = 3.47 \times 10^9$ for Qwen2.5-14B-Instruct (Table 7). With $k = 100$, Theorem 5.4 gives $\varepsilon_{full} \approx \varepsilon_{on}$ at the reported precision for all finite values shown in the figure. The large value of ε_{off} reflects the looseness of the closed-form bound. We use the attack evaluations in Section 7.2 and Section 7.3 to characterize the practical effect of the offline transformation. Reducing ε_{on} from 20 to 14 gives $\varepsilon_{full} \approx 14$, which is still a very weak privacy bound, while lowering C-Eval from 81.71 to 61.81. Further research is necessary to close the remaining gap between this approach and strong privacy guarantees.

7.5 Generalization and Deployability

We next evaluate TWIST across model families and scales and measure its deployment and serving costs.

Generalization across families and scales. Table 4 extends the evaluation to seven models spanning dense and MoE architectures. DeepSeek-V3.1-Terminus is evaluated in `no_think` mode. On the 671B DeepSeek-V3.1-Terminus, MMLU and C-Eval decrease by 1.05 and 2.02 points, with TTRSR at 4.80%. Token recovery varies substantially across models (up to 25.05%), whereas PIIRSR remains below 3% in every case. These benchmarks use the defaults in

Table 4: TWIST utility and privacy across 7 models. Each utility cell reports the change from plaintext → TWIST. The privacy column reports VMA TTRSR and PIIRSR on PUPA. Lower indicates less recovery.

Model	MMLU	C-Eval	HumanEval	PIQA	IFEval	TTRSR / PIIRSR (%)
Llama3-8B	65.91 → 63.50	50.16 → 44.55	55.49 → 57.32	80.74 → 78.94	68.58 → 67.84	2.57 / 0.00
R1-Distill-14B	86.18 → 85.93	78.91 → 79.20	96.34 → 93.90	88.14 → 88.79	72.83 → 72.27	12.36 / 2.37
R1-Distill-32B	88.34 → 87.84	83.50 → 83.61	97.56 → 95.12	88.79 → 88.08	73.94 → 73.57	10.10 / 2.63
Qwen3-14B	87.35 → 87.12	87.64 → 84.57	94.51 → 95.12	89.72 → 90.26	86.14 → 85.40	25.05 / 1.62
Qwen3-32B	89.00 → 87.64	89.61 → 86.24	98.78 → 96.95	90.75 → 89.50	84.29 → 81.15	19.64 / 1.23
Qwen3-MoE-30B-A3B	86.97 → 86.79	84.75 → 84.49	85.37 → 83.54	89.55 → 82.21	80.41 → 78.56	5.91 / 2.43
DeepSeek-V3.1-671B	91.72 → 90.67	87.67 → 85.65	94.51 → 95.12	83.84 → 80.96	86.69 → 84.66	4.80 / 1.12

Table 5: Offline transformation time and online overhead at concurrency 1.

Model	Offline (min)	TTFT (ms)	TPOT (ms)
R1-Distill-14B	3.43	20.56 (+5.5%)	6.72 (−3.0%)
R1-Distill-32B	9.28	31.20 (+4.1%)	9.84 (+1.9%)
Qwen3-MoE-30B-A3B	4.58	24.72 (+3.2%)	7.81 (+2.1%)
DeepSeek-V3.1-671B	482.38	172.04 (+3.6%)	19.63 (+2.6%)

Table 6: TFMA TTRSR and SDA BLEU-4 after observing 7.6×10^6 tokens under three attacker priors.

	Attack setting		TTRSR (%) ↓		BLEU-4 ↓
Attack prior	Public	Private	top-10	top-100	SDA
None	CCI3	Huatuo26M	0.14	1.01	0.01
Domain	MedDialog	Huatuo26M	0.43	3.83	0.55
Distribution	Huatuo26M	Huatuo26M	3.19	16.51	2.10

Section 7.1, except that Llama3-8B uses $\sigma_e = 0.5$, and apply the fixed calibration procedure in Section 6.

Offline transformation cost and online overhead. Table 5 reports the one-time transformation cost and online latency. We measure online latency as time to first token (TTFT) and time per output token (TPOT). Both are reported in milliseconds, with percentage changes relative to plaintext inference shown in parentheses in Table 5. For dense models (14B and 32B), the offline transformation takes 3.43–9.28 minutes. Transforming the 671B MoE model takes 482.38 minutes (≈ 8 hours), but this is a one-time deployment cost amortized over the model lifetime.

At the default expansion size $h = 128$ and concurrency 1, all measured TTFT and TPOT changes remain below 6% in magnitude. On R1-Distill-14B, TTFT increases by 5.5% and measured TPOT decreases by 3.0%. On DeepSeek-V3.1-Terminus at concurrency 4, TTFT and TPOT increase by 8.0% and 4.6%, respectively.

The expansion-size experiment in Appendix D varies h from 128 to 512 on R1-Distill-14B at concurrency 4, with TPOT growth below 10%.

7.6 Token Frequency Leakage

We then consider a longer-horizon threat: frequency leakage when the same token permutation is reused. Single-request recovery does not capture leakage from reusing the same token permutation over a long token sequence. This threat is analogous to classical substitution-cipher attacks, where a fixed symbol mapping leaks statistical structure when observations accumulate over time [2]. With a fixed π, the provider can aggregate counts of provider-visible token IDs and compare them with a prior over plaintext token frequencies. We evaluate two attacks: TFMA estimates token correspondences from frequency statistics, while SDA uses a causal language model to reconstruct plaintext from frequency-ranked ciphertext [2]. TFMA is scored by TTRSR at top-10 and top-100, and SDA by the BLEU-4 score of recovered text.

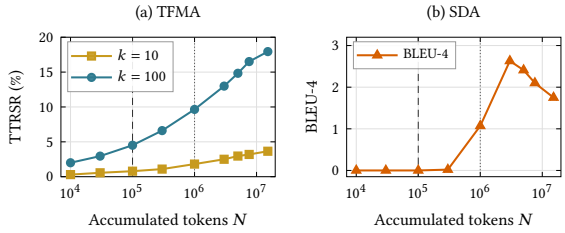


Figure 4: Token-frequency attacks as the number of observed tokens increases, using a fixed secret permutation and the distribution-aware prior from disjoint Huatuo26M-Lite subsets. (a) TFMA TTRSR at top-10 and top-100. (b) SDA BLEU-4.

Table 6 compares attacker priors after observing 7.6×10^6 tokens. **Zero-knowledge** uses CCI3 [69] as the public prior. The private token sequence comes from Huatuo26M-Lite [70]. **Domain-aware** uses MedDialog [8] as the prior. **Distribution-aware** uses disjoint Huatuo26M-Lite subsets. Recovery increases as the attacker prior better matches the private distribution: top-100 TFMA TTRSR rises from 1.01% to 16.51%, and SDA reaches BLEU-4 2.10.

Using the distribution-aware prior, Figure 4 increases the observed-token count from 10^4 to 1.52×10^7. TFMA TTRSR at top-10 rises from 0.30% to 3.65% and TTRSR at top-100 rises from 2.00% to 17.94% over this range. SDA BLEU-4 peaks near 2.63 and then falls to 1.75, so additional observations do not monotonically improve sequence-level reconstruction as measured by BLEU-4. These results indicate that frequency leakage accumulates when the same permutation is reused over long token sequences. Key rotation could cap the number of observations available under any one permutation. How often to rotate is a deployment choice that we leave for future work.

8 Discussion and Limitations

8.1 Scope and Limitations of Formal Guarantees

TWIST is designed for tenant-dedicated BYOM serving and does not aim to protect against all forms of inference leakage. MPC- and FHE-based methods provide stronger cryptographic protection at higher computational cost, while TEE-based approaches rely on trusted hardware. TWIST instead targets compatibility with existing serving infrastructure through offline checkpoint transformation and lightweight client-side secret state. These approaches are complementary, and TWIST can be combined with hardware protection when available. Section 5 formally analyzes the transformed checkpoint and token requests under metrics on token permutations and plaintext sequences. Long-horizon frequency leakage from reusing a fixed permutation is evaluated separately in Section 7.6 and remains outside the closed-form RmDP accounting.

8.2 Potential Threats and Countermeasures

The provider can also infer information from the following signals: **(1) Length leakage.** Prompt and output lengths remain visible to the provider, as in classical analysis of encrypted traffic [37]. Padding or bucketing can reduce this leakage when stronger length protection is required. **(2) Equality-pattern leakage.** Repeated tokens or repeated prompt structures remain identifiable after token permutation, as under deterministic encryption [47]. Such leakage can be mitigated through application-level preprocessing or isolating workloads with highly repetitive templates. **(3) Timing leakage.** Generation latency and request timing may reveal information about user activity, a known leakage of encrypted interactive traffic [57]. Mitigating such leakage requires system-level techniques such as batching or traffic shaping. **(4) Co-occurrence leakage.** Reusing the same permutation across many requests allows the provider to accumulate token co-occurrence structure under a fixed secret, the same type of statistical structure exploited in leakage-abuse attacks on searchable encryption [5]. Session isolation and short log retention can reduce this exposure. **(5) Frequency leakage.** Long token sequences also accumulate token frequency statistics, similar to classical substitution-cipher attacks [2] and frequency analysis against deterministic encryption [47]. Section 7.6 evaluates frequency-based recovery under different attacker priors. Periodic key rotation can limit the observations associated with each permutation.

9 Related Work

9.1 Cryptographic and Hardware Methods

Cryptographic private inference uses MPC, function secret sharing, or homomorphic encryption to evaluate Transformer models [23, 25, 34, 41, 76]. These approaches provide strong confidentiality, but incur substantial computation or communication costs for autoregressive inference. TEE-based systems instead rely on hardware isolation. PipeLLM targets confidential GPUs [59], while earlier systems such as Slalom and SOTER combine CPU trusted execution with untrusted accelerators [55, 64], and ccAI protects accelerator execution through a PCIe security controller [68]. Cascade [61] distributes token positions across inference nodes, reducing reliance on full secure computation but introducing a multi-node separation assumption. In contrast, TWIST targets standard tenant-dedicated serving without requiring cryptographic inference, trusted hardware, or distributed trust.

9.2 Obfuscation-Based Private LLM Inference

Lightweight approaches instead change what the provider observes. Text-side methods sanitize, perturb, or rewrite tokens and prompts [9, 10, 38, 42, 62, 65, 73], while representation-level methods transform embeddings or hidden states [16, 52, 80]. Other designs alter the client-server boundary. Split-and-Denoise [43] and NOIR [49] transmit protected representations, PAPILLON [35] combines local and remote models, and PrivacyRestore [74] sends redacted text with protected restoration information. Prior attacks show that both obfuscated embeddings [40] and deeper internal states [15] can retain recoverable information about the input.

ObfusLM [39] is particularly close to TWIST because it also modifies the deployed model. It permutes the vocabulary and obfuscates the input embeddings and language-model head, while ObfusLM+ applies embedding-dimension obfuscation across the model to protect intermediate representations. The resulting model, however, requires fine-tuning to adapt to the obfuscated embeddings. TWIST instead derives module-specific transformations around the algebra of attention, FFNs, normalization, residual paths, and MoE routing, and can be deployed without retraining or serving-engine modifications. TWIST also analyzes the one-time checkpoint release and the request-level token mechanism together under RmDP.

9.3 Differential Privacy

Differential privacy has been explored for protecting text and LLM-related data at several levels. Local and word-level mechanisms randomize text on the client side [18, 44], while other work applies DP to private examples used in prompt learning and few-shot generation [17, 60]. TWIST considers a different secret space consisting of token permutations and consistently renamed token sequences. We therefore build on metric DP [6, 30, 71], which measures privacy with respect to distances between secrets, together with Rényi DP [46] for privacy accounting. Our formulation further relates to concentrated geo-privacy [36] through the correspondence described in Definition 5.1.

10 Conclusion

We present TWIST, a practical system that protects inputs and outputs in tenant-dedicated LLM serving. TWIST uses joint model-token transformation: the client applies the online token permutation, while the provider runs a transformed model checkpoint using standard LLM serving infrastructure. We account for the analyzed channel under RmDP, where the joint design yields a smaller composed budget than a data-only baseline under the same online mechanism. Our empirical evaluation, including models up to 671B parameters, shows reduced token recovery while preserving utility. By bringing joint model-token protection to commodity LLM serving without requiring specialized hardware, we hope TWIST takes a step toward practical private LLM deployments.

Acknowledgments

We thank the anonymous reviewers and our shepherd for their constructive feedback and guidance. We also thank Hanning Xia, Prof. Yunlong Mao, and Prof. Sheng Zhong from Nanjing University for their helpful discussions and feedback. This work was supported in part by the Research Grants Council of the Hong Kong Special Administrative Region, China, under Grants CityU 11218322, 11219524, R6021-20F, R1012-21, RFS21221S04, C2004-21G, C1029-22G, and C6015-23G, and by the Innovation and Technology Commission (ITC) under the Mainland-Hong Kong Joint Funding Scheme (MHKJFS), Grant MHP/135/23. This work was also supported in part by the InnoHK initiative of the Government of the HKSAR, the Laboratory for AI-Powered Financial Technologies (AIFT), and the Guangdong and Hong Kong Universities "1+1+1" Joint Research Collaboration Scheme. Grok was used for grammar correction and minor editorial polishing. All technical content and evaluation artifacts were reviewed and verified by the authors.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proc. of EMNLP*, 2023.
[2] Nada Aldarrab and Jonathan May. Can Sequence-to-Sequence Models Crack Substitution Ciphers? In *Proc. of ACL-IJCNLP*, 2021.
[3] Amazon Web Services. n.d. Deploy a Custom Model on Amazon SageMaker AI. AWS Documentation. Retrieved September 13, 2026 from https://docs.aws.amazon.com/sagemaker/latest/dg/deploy-trained-model.html
[4] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, et al. PIQA: Reasoning about Physical Commonsense in Natural Language. In *Proc. of AAAI*, 2020.
[5] David Cash, Paul Grubbs, Jason Perry, and Thomas Ristenpart. Leakage-Abuse Attacks Against Searchable Encryption. In *Proc. of CCS*, 2015.
[6] Konstantinos Chatzikokolakis, Miguel E. Andrés, Nicolás Emilio Bordenabe, et al. Broadening the Scope of Differential Privacy Using Metrics. In *Proc. of PETS*, 2013.
[7] Mark Chen, Jerry Tworek, Heewoo Jun, et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374
[8] Shu Chen, Zeqian Ju, Xiangyu Dong, et al. 2020. MedDialog: A Large-scale Medical Dialogue Dataset. arXiv:2004.03329v1
[9] Sai Chen, Fengran Mo, Yanhao Wang, et al. A Customized Text Sanitization Mechanism with Differential Privacy. In *Findings of ACL*, 2023.
[10] Amrita Roy Chowdhury, David Glukhov, Divyam Anshumaan, et al. 2025. Pr$\epsilon\epsilon$mpt: Sanitizing Sensitive Prompts for LLMs. arXiv:2504.05147
[11] DeepSeek-AI. 2025. DeepSeek-V3.1-Terminus. https://huggingface.co/deepseek-ai/DeepSeek-V3.1-Terminus
[12] DeepSeek-AI, Daya Guo, Dejian Yang, et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948
[13] DeepSeek-AI, Aixin Liu, Bei Feng, et al. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434
[14] DeepSeek-AI, Aixin Liu, Bei Feng, et al. 2024. DeepSeek-V3 Technical Report. arXiv:2412.19437
[15] Tian Dong, Yan Meng, Shaofeng Li, et al. Depth Gives a False Sense of Privacy: LLM Internal States Inversion. In *Proc. of USENIX Security*, 2025.
[16] Minxin Du, Xiang Yue, Sherman S. M. Chow, et al. DP-Forward: Fine-tuning and Inference on Language Models with Differential Privacy in Forward Pass. In *Proc. of CCS*, 2023.
[17] Haonan Duan, Adam Dziedzic, Nicolas Papernot, et al. Flocks of Stochastic Parrots: Differentially Private Prompt Learning for Large Language Models. In *Proc. of NeurIPS*, 2023.
[18] John C. Duchi, Michael I. Jordan, and Martin J. Wainwright. Local Privacy and Statistical Minimax Rates. In *Proc. of FOCS*, 2013.
[19] Cynthia Dwork. Differential Privacy. In *Proc. of ICALP*, 2006.
[20] Ege Erdoğan, Alptekin Küpçü, and A. Ercüment Çiçek. UnSplit: Data-Oblivious Model Inversion, Model Stealing, and Label Inference Attacks against Split Learning. In *Proc. of WPES*, 2022.
[21] Google Cloud. n.d. Deploy a Model to an Endpoint. Gemini Enterprise Agent Platform Documentation. Retrieved September 13, 2026 from https://docs.cloud.google.com/gemini-enterprise-agent-platform/machine-learning/general/deployment
[22] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783
[23] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, et al. 2024. SIGMA: Secure GPT Inference with Function Secret Sharing. *Proc. Priv. Enhancing Technol.* 2024, 4 (2024), 61–79.
[24] Otkrist Gupta and Ramesh Raskar. 2018. Distributed Learning of Deep Neural Network over Multiple Agents. *Journal of Network and Computer Applications* 116 (2018), 1–8.
[25] Meng Hao, Hongwei Li, Hanxiao Chen, et al. Iron: Private Inference on Transformers. In *Proc. of NeurIPS*, 2022.
[26] Dan Hendrycks, Collin Burns, Steven Basart, et al. Measuring Massive Multitask Language Understanding. In *Proc. of ICLR*, 2021.
[27] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-Rank Adaptation of Large Language Models. In *Proc. of ICLR*, 2022.
[28] Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, et al. C-Eval: A Multi-Level Multi-Discipline Chinese Evaluation Suite for Foundation Models. In *Proc. of NeurIPS*, 2023.
[29] Hugging Face. n.d. Inference Endpoints (Dedicated). Hugging Face. Retrieved September 13, 2026 from https://huggingface.co/docs/inference-endpoints/index
[30] Jacob Imola, Amrita Roy Chowdhury, and Kamalika Chaudhuri. Metric Differential Privacy at the User-Level via the Earth-Mover's Distance. In *Proc. of CCS*, 2024.
[31] Taku Kudo and John Richardson. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. In *Proc. of EMNLP (System Demonstrations)*, 2018.
[32] Kai Kugler, Simon Münker, Johannes Höhmann, et al. 2021. InvBERT: Reconstructing Text from Contextualized Word Embeddings by inverting the BERT pipeline. arXiv:2109.10104
[33] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proc. of SOSP*, 2023.
[34] Dacheng Li, Rulin Shao, Hongyi Wang, et al. MPCFormer: Fast, Performant and Private Transformer Inference with MPC. In *Proc. of ICLR*, 2023.
[35] Siyan Li, Vethavikashini Chithrra Raghuram, Omar Khattab, et al. PAPILLON: Privacy Preservation from Internet-based and Local Language Model Ensembles. In *Proc. of NAACL*, 2025.
[36] Yuting Liang and Ke Yi. Concentrated Geo-Privacy. In *Proc. of CCS*, 2023.
[37] Marc Liberatore and Brian Neil Levine. Inferring the Source of Encrypted HTTP Connections. In *Proc. of CCS*, 2006.
[38] Sam Lin, Wenyue Hua, Zhenting Wang, et al. EmojiPrompt: Generative Prompt Obfuscation for Privacy-Preserving Communication with Cloud-based LLMs. In *Proc. of NAACL*, 2025.
[39] Yu Lin, Ruining Yang, Yunlong Mao, et al. ObfusLM: Privacy-preserving Language Model Service against Embedding Inversion Attacks. In *Proc. of ACL*, 2025.
[40] Yu Lin, Qizhi Zhang, Quanwei Cai, et al. An Inversion Attack Against Obfuscated Embedding Matrix in Language Model Inference. In *Proc. of EMNLP*, 2024.
[41] Wen-jie Lu, Zhicong Huang, Zhen Gu, et al. BumbleBee: Secure Two-party Inference Framework for Large Transformers. In *Proc. of NDSS*, 2025.
[42] Hongru Ma, Wenpeng Lu, Yanjie Liang, et al. ALSA: Context-Sensitive Prompt Privacy Preservation in Large Language Models. In *Proc. of KDD*, 2025.
[43] Peihua Mai, Ran Yan, Zhe Huang, et al. Split-and-Denoise: Protect Large Language Model Inference with Local Differential Privacy. In *Proc. of ICML*, 2024. https://proceedings.mlr.press/v235/mai24a.html
[44] Justus Mattern, Benjamin Weggenmann, and Florian Kerschbaum. The Limits of Word Level Differential Privacy. In *Findings of NAACL*, 2022.
[45] Microsoft. 2026. Deploy and Score a Machine Learning Model by Using an Online Endpoint. Microsoft Learn. Retrieved September 13, 2026 from https://learn.microsoft.com/en-us/azure/machine-learning/how-to-deploy-online-endpoints
[46] Ilya Mironov. Rényi Differential Privacy. In *Proc. of CSF*, 2017.
[47] Muhammad Naveed, Seny Kamara, and Charles V. Wright. Inference Attacks on Property-Preserving Encrypted Databases. In *Proc. of CCS*, 2015.
[48] Lucien K. L. Ng, Sherman S. M. Chow, Anna P. Y. Woo, et al. Goten: GPU-Outsourcing Trusted Execution of Neural Network Training. In *Proc. of AAAI*, 2021.
[49] Khoa Nguyen, Khiem Ton, NhatHai Phan, et al. 2026. NOIR: Privacy-Preserving Generation of Code with Open-Source LLMs. arXiv:2601.16354
[50] Dario Pasquini, Giuseppe Ateniese, and Massimo Bernaschi. Unleashing the Tiger: Inference Attacks on Split Learning. In *Proc. of CCS*, 2021.
[51] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models! https://qwenlm.github.io/blog/qwen2.5/
[52] Jay Roberts, Kyle Mylonakis, Sidhartha Roy, et al. 2025. Learning Obfuscations Of LLM Embedding Sequences: Stained Glass Transform. arXiv:2506.09452
[53] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units. In *Proc. of ACL*, 2016.
[54] Noam Shazeer. 2019. Fast Transformer Decoding: One Write-Head is All You Need. arXiv:1911.02150
[55] Tianxiang Shen, Ji Qi, Jianyu Jiang, et al. SOTER: Guarding Black-box Inference for General Neural Networks at the Edge. In *Proc. of USENIX ATC*, 2022.

[56] Richard Socher, Alex Perelygin, Jean Wu, et al. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proc. of EMNLP*, 2013.

[57] Dawn Xiaodong Song, David Wagner, and Xuqing Tian. Timing Analysis of Keystrokes and Timing Attacks on SSH. In *Proc. of USENIX Security*, 2001.

[58] Jianlin Su, Murtadha Ahmed, Yu Lu, et al. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing* 568 (2024), 127063.

[59] Yifan Tan, Cheng Tan, Zeyu Mi, et al. PipeLLM: Fast and Confidential Large Language Model Services with Speculative Pipelined Encryption. In *Proc. of ASPLOS*, 2025.

[60] Xinyu Tang, Richard Shin, Huseyin A. Inan, et al. Privacy-Preserving In-Context Learning with Differentially Private Few-Shot Generation. In *Proc. of ICLR*, 2024.

[61] Rahul Krishna Thomas, Louai Zahran, Erica Choi, et al. Hidden No More: Attacking and Defending Private Third-Party LLM Inference. In *Proc. of ICML*, 2025. https://proceedings.mlr.press/v267/thomas25b.html

[62] Meng Tong, Kejiang Chen, Jie Zhang, et al. 2025. InferDPT: Privacy-Preserving Inference for Closed-Box Large Language Models. *IEEE Trans. Dependable Secur. Comput.* 22, 5 (2025), 4625–4640.

[63] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971

[64] Florian Tramèr and Dan Boneh. Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware. In *Proc. of ICLR*, 2019.

[65] Saiteja Utpala, Sara Hooker, and Pin-Yu Chen. Locally Differentially Private Document Generation Using Zero Shot Prompting. In *Findings of EMNLP*, 2023.

[66] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need. In *Proc. of NeurIPS*, 2017.

[67] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, et al. 2018. Split Learning for Health: Distributed Deep Learning without Sharing Raw Patient Data. arXiv:1812.00564

[68] Chenxu Wang, Danqing Tang, Changxu Ci, et al. ccAI: A Compatible and Confidential System for AI Computing. In *Proc. of MICRO*, 2025.

[69] Liangdong Wang, Bo-Wen Zhang, Chengwei Wu, et al. 2024. CCI3.0-HQ: a large-scale Chinese dataset of high quality designed for pre-training large language models. arXiv:2410.18505

[70] Xidong Wang, Jianquan Li, Shunian Chen, et al. Huatuo-26M, a Large-scale Chinese Medical QA Dataset. In *Findings of NAACL*, 2025.

[71] Xinpeng Xie, Chenyang Yu, Yan Huang, et al. 2025. A Decade of Metric Differential Privacy: Advancements and Applications. arXiv:2502.08970

[72] An Yang, Anfeng Li, Baosong Yang, et al. 2025. Qwen3 Technical Report. arXiv:2505.09388

[73] Xiang Yue, Minxin Du, Tianhao Wang, et al. Differential Privacy for Text Analytics via Natural Text Sanitization. In *Findings of ACL-IJCNLP*, 2021.

[74] Ziqian Zeng, Jianwei Wang, Junyao Yang, et al. PrivacyRestore: Privacy-Preserving Inference in Large Language Models via Privacy Removal and Restoration. In *Proc. of ACL*, 2025.

[75] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. In *Proc. of NeurIPS*, 2019.

[76] Jiawen Zhang, Xinpeng Yang, Lipeng He, et al. Secure Transformer Inference Made Non-interactive. In *Proc. of NDSS*, 2025.

[77] Ziqi Zhang, Chen Gong, Yifeng Cai, et al. No Privacy Left Outside: On the (In-)Security of TEE-Shielded DNN Partition for On-Device ML. In *Proc. of IEEE S&P*, 2024.

[78] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al. SGLang: Efficient Execution of Structured Language Model Programs. In *Proc. of NeurIPS*, 2024.

[79] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, et al. 2023. Instruction-Following Evaluation for Large Language Models. arXiv:2311.07911

[80] Xin Zhou, Yi Lu, Ruotian Ma, et al. TextObfuscator: Making Pre-trained Language Model a Privacy Protector via Obfuscating Word Representations. In *Findings of ACL*, 2023.

A Joint-Transformation Framework and Composition Proofs

This framework formalizes the joint model-token transformation in Section 4. This appendix specifies the spaces, decoding maps, and error assumptions used in the composition argument. It then derives the propagation of module errors through residual blocks, logits, and autoregressive outputs.

A.1 Definition

Let $f : X \times \Theta \to Y$ be a module with input, parameter, and output spaces X, Θ, Y. Its transformed counterpart is $\widetilde{f} : \widetilde{X} \times \widetilde{\Theta} \to \widetilde{Y}$. A joint transformation consists of five maps,

$$C = (\phi_X, \phi_\Theta, \phi_Y, \psi_Y, \widetilde{f}),$$

where $\phi_X : X \to \widetilde{X}$, $\phi_\Theta : \Theta \to \widetilde{\Theta}$, $\phi_Y : Y \to \widetilde{Y}$, and $\psi_Y : \widetilde{Y} \to Y$. The transformed spaces have specified metrics. For each (x, θ) in a stated input and parameter domain, define the random local error

$$\Delta_C(x, \theta) = d_{\widetilde{Y}}\big(\widetilde{f}(\phi_X(x), \phi_\Theta(\theta)), \phi_Y(f(x, \theta))\big).$$

The expected commutation condition is

$$\sup_{(x,\theta)} \mathbb{E}[\Delta_C(x, \theta)] \le e_C,$$

with expectation over the jointly sampled transformation randomness. The decoding identity is $\psi_Y \circ \phi_Y = \mathrm{id}_Y$. Compatibility at a shared boundary means equality of the realized maps at that boundary, including their shared randomness.

Each composition below uses a deterministic Lipschitz bound that holds uniformly over the stated domain and the transformation realizations under consideration. Alternatively, one may condition on a realized transformed model checkpoint and use its corresponding local errors and Lipschitz constants. The conditional bound can subsequently be averaged, retaining products of random constants inside the expectation. This distinction specifies the expectation needed when noise and key matrices also affect the Lipschitz constants.

Decoded error. If ψ_Y is L_ψ-Lipschitz, then

$$\mathbb{E}\Big[d_Y\big(\psi_Y(\widetilde{f}(\phi_X(x), \phi_\Theta(\theta))), f(x, \theta)\big)\Big] \le L_\psi e_C.$$

Indeed, apply the Lipschitz inequality to the two transformed outputs and use $\psi_Y(\phi_Y(f(x, \theta))) = f(x, \theta)$. For a row-vector encoding $\phi_Y(y) = y\widehat{P}$ with $\widehat{P}\widehat{Q} = I$, decoding is $\psi_Y(\widetilde{y}) = \widetilde{y}\widehat{Q}$ and $L_\psi = \|\widehat{Q}\|_2$ under the Euclidean norm. A token permutation and its inverse are isometries under tokenwise equality distance.

A.2 Data-Only Obfuscation as a Special Case

Data-only obfuscation is obtained by taking $\widetilde{X} = X$, $\widetilde{\Theta} = \Theta$, $\widetilde{Y} = Y$, $\phi_\Theta = \mathrm{id}_\Theta$, $\phi_Y = \psi_Y = \mathrm{id}_Y$, and $\widetilde{f} = f$. Only the input map ϕ_X remains. A joint transformation also permits a model transformation and a compatible output encoding. This inclusion relates the two design spaces. A quantitative privacy comparison depends on the mechanisms and distributions analyzed in Section 5.

A.3 Sequential Composition

Theorem A.1 (Sequential composition). *Let C_1 transform $f : X \times \Theta \to Y$ and C_2 transform $g : Y \times \Xi \to Z$, with the same realized intermediate map ϕ_Y. If $\widetilde{g}$ is uniformly M_g-Lipschitz in its input, their composition transforms $h(x, \theta, \xi) = g(f(x, \theta), \xi)$ and satisfies*

$$e_{C_2 \circ C_1} \le M_g e_{C_1} + e_{C_2}.$$

Proof. Write $y = f(x, \theta)$, $\widetilde{y} = \widetilde{f}(\phi_X(x), \phi_\Theta(\theta))$, and $\widetilde{\xi} = \phi_\Xi(\xi)$. The composed output encoding and decoder are ϕ_Z and ψ_Z. The

triangle inequality gives

$$\begin{aligned} & d_{\widetilde{Z}}\big(\widetilde{g}(\widetilde{y},\widetilde{\xi}),\phi_Z(g(y,\xi))\big) \\ & \quad \le d_{\widetilde{Z}}\big(\widetilde{g}(\widetilde{y},\widetilde{\xi}),\widetilde{g}(\phi_Y(y),\widetilde{\xi})\big) \\ & \qquad + d_{\widetilde{Z}}\big(\widetilde{g}(\phi_Y(y),\widetilde{\xi}),\phi_Z(g(y,\xi))\big) \\ & \quad \le M_g\Delta_{C_1}(x,\theta) + \Delta_{C_2}(y,\xi). \end{aligned}$$

Taking expectations and using the uniform local bounds proves the inequality. The identity $\psi_Z \circ \phi_Z = \mathrm{id}_Z$ supplies the decoding condition. □

A.4 Parallel Composition

Theorem A.2 (Parallel composition). *Let C_1 transform $f : X \times \Theta \to Y$ and C_2 transform $g : X \times \Xi \to Z$, with the same realized input map. Equip the transformed product space with a metric satisfying*

$$d_{\widetilde{Y}\times\widetilde{Z}}((y,z),(y',z')) \le d_{\widetilde{Y}}(y,y') + d_{\widetilde{Z}}(z,z').$$

Then the product transformation of $h(x,\theta,\xi) = (f(x,\theta), g(x,\xi))$ satisfies

$$e_{C_1\|C_2} \le e_{C_1} + e_{C_2}.$$

Proof. Apply the product-distance inequality to the two output discrepancies and take expectations. The product decoder obeys

$$(\psi_Y,\psi_Z)\circ(\phi_Y,\phi_Z) = (\mathrm{id}_Y,\mathrm{id}_Z).$$

The argument uses linearity of expectation and allows dependent branch randomness. □

A.5 Summation Composition

Theorem A.3 (Summation composition). *Let C_1 and C_2 transform $f : X\times\Theta \to Y$ and $g : X\times\Xi \to Y$ with the same realized input and output encodings and decoder. Assume $Y, \widetilde{Y}$ are additive groups, ϕ_Y is additive, and $d_{\widetilde{Y}}$ is translation-invariant. The sum transformation of $h(x,\theta,\xi) = f(x,\theta) + g(x,\xi)$ satisfies*

$$e_{C_1+C_2} \le e_{C_1} + e_{C_2}.$$

Proof. For $a_1, a_2, b_1, b_2 \in \widetilde{Y}$,

$$d_{\widetilde{Y}}(a_1+b_1, a_2+b_2) \le d_{\widetilde{Y}}(a_1,a_2) + d_{\widetilde{Y}}(b_1,b_2).$$

This follows by inserting a_2+b_1 and applying translation invariance. Use $\phi_Y(f+g) = \phi_Y(f)+\phi_Y(g)$ and take expectations. The common output encoding and decoder retain $\psi_Y \circ \phi_Y = \mathrm{id}_Y$. □

A.6 End-to-End Aggregation

Theorem A.4 (Propagation through the model). *Consider an embedding, L decoder blocks, and an output head with compatible realized boundary maps. Suppose their expected local commutation errors are bounded by $e_0, e_1, \dots, e_L, e_{\mathrm{h}}$ on the stated domain. Let $M_1, \dots, M_L, M_{\mathrm{h}}$ be uniform Lipschitz constants for the transformed decoder blocks and head. The composite logit transformation has expected error at most*

$$e_{\mathrm{logit}} \le M_{\mathrm{h}}\left(e_0\prod_{j=1}^{L} M_j + \sum_{i=1}^{L} e_i \prod_{j=i+1}^{L} M_j\right) + e_{\mathrm{h}}.$$

Empty products equal one.

Proof. Let δ_i bound the discrepancy after block i. Sequential composition gives $\delta_0 \le e_0$ and $\delta_i \le M_i\delta_{i-1} + e_i$. Expanding this recurrence gives the parenthesized expression. Applying the head gives the displayed result. Parallel and summation composition establish the corresponding local block error when attention branches, expert branches, or residual additions share their required boundary maps. The final decoder supplies the inverse-map condition. □

A pre-normalized residual block. Write the plaintext block as

$$u = x + A(N_1(x)), \qquad B(x) = u + F(N_2(u)).$$

Use $e_{N_1}, e_A, e_{N_2}, e_F$ for the four local errors and $M_{N_1}, M_A, M_{N_2}, M_F$ for the transformed Lipschitz constants. The two branch errors are bounded by

$$a = M_A e_{N_1} + e_A, \qquad b = M_F e_{N_2} + e_F.$$

Applying summation to each identity residual path and then sequential composition gives

$$e_B \le (1 + M_F M_{N_2})a + b,$$

and a block Lipschitz constant is

$$M_B \le (1 + M_F M_{N_2})(1 + M_A M_{N_1}).$$

Thus residual paths contribute the identity terms, and errors introduced early in the network are multiplied by the later sensitivity factors. Each module's local error must be bounded in the computation metric used here. The Gaussian privacy calculation uses a separate divergence metric.

A.7 From Logits to Generated Tokens

For greedy decoding, let the largest and second-largest plaintext logits differ by $m > 0$. If the decoded transformed logits differ from the plaintext logits by at most δ in the maximum norm and $2\delta < m$, the largest-logit token is unchanged. This follows by bounding the leading logit from below by its original value minus δ and every competitor from above by its value plus δ. A numerical logit bound therefore supports a token decision through its margin.

For full-softmax sampling at temperature $\tau > 0$, write $p = \mathrm{softmax}(z/\tau)$ and $q = \mathrm{softmax}(z'/\tau)$. The softmax Jacobian satisfies $\|J(z)v\|_1 \le 2\|v\|_\infty$. Integration along the segment from z to z' gives

$$\mathrm{TV}(p,q) \le \min\{1, \|z - z'\|_\infty/\tau\}.$$

Top-k and top-p sampling additionally depend on the selected support. Their analysis requires the ranking and cumulative-probability margins at the truncation boundary.

For autoregressive generation, suppose the conditional next-token distributions differ in total variation by at most η_t at step t whenever the previous generated histories agree. Couple each pair of next-token draws maximally while the histories agree. The probability of any disagreement in the first T generated tokens is at most

$$\min\left\{1, \sum_{t=1}^{T}\eta_t\right\}.$$

This follows by summing the probabilities of the first disagreement at each step. The token permutation is inverted at the vocabulary boundary throughout this comparison. The online exponential mechanism is an additional randomized input transformation, whose measured utility effect is reported in Section 7.4.

B Rényi-Metric Proofs

This appendix gives alignment results for real matrices, the Gaussian and exponential bounds, and the offline proof under the common release map in Theorem 5.3. For real matrices, the inner product is $\langle X, Y\rangle = \mathrm{tr}(X^\top Y)$ and all matrix norms without a subscript are Frobenius norms. Write E_{ij} for the elementary matrix and $e_{ij} = E_{ij} - E_{ji}$ for the skew-symmetric basis element. Thus $\|e_{ij}\|_F^2 = 2$. Sums over skew-basis indices without explicit limits use $1 \le i < j \le n$ for n-dimensional matrices.

B.1 Quadratic Forms on Real Skew-Symmetric Matrices

Lemma B.1 (Quadratic Forms on Skew-Symmetric Matrices). *Consider quadratic forms on $\mathfrak{so}(n) := \{A \in M_n; A^t = -A\}$:*

(1) *For a symmetric matrix $A \in M_n$, the function $f(X) := \mathrm{tr}\, X^T AX$ has gradient $\nabla f(X) = AX + XA$, and*

$$f(X) \le \frac{\lambda_1 + \lambda_2}{2}\|X\|^2,$$

where λ_1, λ_2 are the largest and second-largest eigenvalues of A.

(2) *For $A \in M_{m,n}$, the function $f(X) := \|AX\|_F^2$ has gradient $\nabla f(X) = A^TAX + XA^TA$, and*

$$f(X) \le \frac{\lambda_1^2 + \lambda_2^2}{2}\|X\|^2,$$

where λ_1, λ_2 are the largest and second-largest singular values of A.

(3) *For a square matrix $A \in M_n$, the function $f(X) := \mathrm{tr}\, AX$ has gradient*

$$\nabla f(X) = \frac{1}{2}(A^T - A).$$

(4) *For $A \in M_{m,n}$, the function $f(X) := \|AXA^T\|_F^2$ has gradient*

$$\nabla f(X) = 2A^TAXA^TA.$$

Proof. (1)

$$f(X) := \mathrm{tr}\, X^TAX = \frac{1}{2}\mathrm{tr}(X^TAX + XX^TA) = \frac{1}{2}\mathrm{tr}(X^TAX + X^TXA) = \frac{1}{2}\mathrm{tr}\, X^T(AX + XA).$$

Thus $\nabla f(X) = AX + XA$.
Since A is symmetric, it admits an orthogonal similarity decomposition $A = Q^T\Lambda Q$, so

$$\nabla f(X) = Q^T\Lambda QX + XQ^T\Lambda Q.$$

Let $\tilde{e}_{i,j} = Q^T e_{i,j} Q$, then

$$\begin{aligned}\nabla f(\tilde{e}_{i,j}) &= Q^T\Lambda e_{i,j}Q + Q^T e_{i,j}\Lambda Q\\ &= Q^T(\Lambda e_{i,j} + e_{i,j}\Lambda)Q\\ &= Q^T(\lambda_i E_{i,j} - \lambda_j E_{j,i} + E_{i,j}\lambda_j - E_{j,i}\lambda_i)Q\\ &= Q^T(\lambda_i + \lambda_j)e_{i,j}Q = (\lambda_i + \lambda_j)\tilde{e}_{i,j}.\end{aligned}$$

Therefore,

$$\begin{aligned}f(X) &= \frac{1}{2}\langle X, \nabla f(X)\rangle = \frac{1}{2}\left\langle \sum_{i,j} x_{i,j}\tilde{e}_{i,j}, \sum_{i,j} x_{i,j}(\lambda_i + \lambda_j)\tilde{e}_{i,j}\right\rangle\\ &= \frac{1}{2}\sum_{i,j}(\lambda_i + \lambda_j)x_{i,j}^2\|\tilde{e}_{i,j}\|^2 = \sum_{i,j}(\lambda_i + \lambda_j)x_{i,j}^2\\ &\le (\lambda_1 + \lambda_2)\sum_{i,j} x_{i,j}^2 = \frac{\lambda_1 + \lambda_2}{2}\|X\|^2.\end{aligned}$$

(2) Note that $f(X) = \mathrm{tr}\, X^TA^TAX$, which is a special case of (1) with A replaced by A^TA.

(3)

$$\begin{aligned}f(X) := \mathrm{tr}\, AX &= \frac{1}{2}\mathrm{tr}(AX + X^TA^T)\\ &= \frac{1}{2}\mathrm{tr}(XA + X^TA^T)\\ &= \frac{1}{2}\mathrm{tr}\, X^T(A^T - A).\end{aligned}$$

Hence $\nabla f(X) = \frac{1}{2}(A^T - A)$.

(4)

$$\begin{aligned}f(X) &:= \|AXA^T\|_F^2\\ &= \mathrm{tr}(AX^TA^TAXA^T)\\ &= \mathrm{tr}(X^TA^TAXA^TA).\end{aligned}$$

Differentiating directly gives $\nabla f(X) = 2A^TAXA^TA$.

□

Theorem B.1. *(Quadratic Forms on Skew-Symmetric Matrices) For $A \in \mathbb{R}^{n\times d}$ with $n > d$, and skew-symmetric matrices $X \in \mathfrak{so}(n)$, $c > 0$, define*

$$f(X) := \min_{Y\in\mathfrak{so}(d)} \|XA - AY\|_F^2 + c\|Y\|^2.$$

Then the following hold:

(1) *The minimized form is*

$$f(X) = \|XA\|_F^2 - 2\langle A^TXA, \mathcal{L}^{-1}(A^TXA)\rangle,$$

where $\mathcal{L} : \mathfrak{so}(d) \to \mathfrak{so}(d)$ is the self-adjoint operator defined by

$$\frac{1}{2}\langle \mathcal{L}(Y), Y\rangle = \|AY\|_F^2 + c\|Y\|^2,$$

i.e., $\mathcal{L}(Y) := (A^TA + cI_d)Y + Y(A^TA + cI_d)$.

(2) $\nabla f(X) = XAA^T + AA^TX - 4A\mathcal{L}^{-1}(A^TXA)A^T$.

(3) $f\left(P^T\sum_{1\le i<j\le n} x_{i,j}e_{i,j}P\right) = \sum_{1\le i<j\le n}\mu_{i,j}x_{i,j}^2$, *where P is the orthogonal matrix from the singular value decomposition (SVD) $A = P^T\begin{pmatrix}\Lambda\\0\end{pmatrix}Q$, and*

$$\mu_{i,j} = \left[(\lambda_i^2 + \lambda_j^2) - \frac{4\lambda_i^2\lambda_j^2}{\lambda_i^2 + \lambda_j^2 + 2c}\right],$$

with $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ being the n singular values of A (only the first d may be non-zero). In particular,

$$f(X) \leq \frac{1}{2} \max_{1 \leq i < j \leq n} \mu_{i,j} \|X\|^2.$$

(4) $f(X) \leq \frac{1}{2} \max(\mu_{1,2}, \mu_{1,n}) \|X\|^2 = \frac{1}{2} \max(\mu_{1,2}, \lambda_1^2) \|X\|^2$.

Proof. (1) Let

$$\begin{aligned} F(X,Y) &:= \|XA - AY\|_F^2 + c\|Y\|^2 \\ &= \|XA\|_F^2 - 2 \operatorname{tr} Y^T A^T X A + \operatorname{tr} Y^T (A^T A + cI_d) Y. \end{aligned}$$

Then

$$\begin{aligned} F(X,Y) &= \|XA\|_F^2 - 2\langle A^T XA, Y\rangle + \frac{1}{2}\langle \mathcal{L}Y, Y\rangle \\ &= \|XA\|_F^2 + \frac{1}{2}\left\langle \mathcal{L}\left(Y - 2\mathcal{L}^{-1}(A^T XA)\right), Y - 2\mathcal{L}^{-1}(A^T XA)\right\rangle \\ &\quad - 2\langle A^T XA, \mathcal{L}^{-1}(A^T XA)\rangle. \end{aligned}$$

For fixed X, $F(X,Y)$ attains its minimum if and only if $Y = 2\mathcal{L}^{-1}(A^T XA)$, with the minimum value

$$f(X) = \|XA\|_F^2 - 2\langle A^T XA, \mathcal{L}^{-1}(A^T XA)\rangle.$$

(2)

$$\begin{aligned} f(X) &= \|XA\|_F^2 - 2\langle A^T XA, \mathcal{L}^{-1}(A^T XA)\rangle \\ &= \operatorname{tr} XAA^T X^T - 2 \operatorname{tr} A^T X^T A \mathcal{L}^{-1}(A^T XA) \\ &= \frac{1}{2} \operatorname{tr} X^T (XAA^T + AA^T X) - 2 \operatorname{tr} X^T A \mathcal{L}^{-1}(A^T XA) A^T \\ &= \frac{1}{2} \left\langle X, XAA^T + AA^T X - 4A\mathcal{L}^{-1}(A^T XA)A^T \right\rangle. \end{aligned}$$

Thus

$$\nabla f(X) = XAA^T + AA^T X - 4A\mathcal{L}^{-1}(A^T XA)A^T.$$

(3) Let $A = P^T \begin{pmatrix} \Lambda \\ 0 \end{pmatrix} Q$, where P, Q are orthogonal matrices and $\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_d)$ is diagonal. Define

$$\begin{aligned} \tilde{e}_{i,j} &= P^T e_{i,j} P \qquad \text{for } 1 \leq i < j \leq n, \\ \eta_{i,j} &:= Q^T e_{i,j} Q \qquad \text{for } 1 \leq i < j \leq d. \end{aligned}$$

Set $\lambda_i = 0$ for $i > d$, and set $\eta_{i,j} = 0$ when $j > d$. The elementary and skew-basis matrices in each expression have the dimension required by their surrounding factors. Let $D_n = \operatorname{diag}(\lambda_1^2, \ldots, \lambda_n^2)$. Since $AA^\top = P^\top D_n P$,

$$\begin{aligned} &\tilde{e}_{i,j} AA^\top + AA^\top \tilde{e}_{i,j} \\ &\quad = P^\top (e_{i,j} D_n + D_n e_{i,j}) P \\ &\quad = (\lambda_i^2 + \lambda_j^2) \tilde{e}_{i,j}. \end{aligned}$$

The SVD also gives

$$A^\top \tilde{e}_{i,j} A = \lambda_i \lambda_j \eta_{i,j}, \qquad A\eta_{i,j} A^\top = \lambda_i \lambda_j \tilde{e}_{i,j}.$$

Both identities are zero when $j > d$. For $j \leq d$, they follow by multiplying the two nonzero elementary entries of $e_{i,j} = E_{i,j} - E_{j,i}$ by the corresponding singular values. Similarly,

$$\mathcal{L}(\eta_{i,j}) = (\lambda_i^2 + \lambda_j^2 + 2c)\eta_{i,j},$$

and hence

$$\mathcal{L}^{-1}(\eta_{i,j}) = \frac{\eta_{i,j}}{\lambda_i^2 + \lambda_j^2 + 2c}.$$

Substitution in the gradient formula gives

$$\begin{aligned} \nabla f(\tilde{e}_{i,j}) &= \left(\lambda_i^2 + \lambda_j^2 - \frac{4\lambda_i^2 \lambda_j^2}{\lambda_i^2 + \lambda_j^2 + 2c}\right) \tilde{e}_{i,j} \\ &= \mu_{i,j} \tilde{e}_{i,j}. \end{aligned}$$

The basis elements are orthogonal and have squared Frobenius norm two. Therefore, for $X = \sum_{1 \leq i < j \leq n} x_{i,j} \tilde{e}_{i,j}$,

$$\begin{aligned} f(X) &= \frac{1}{2}\left\langle \sum_{1 \leq i < j \leq n} x_{i,j} \tilde{e}_{i,j}, \nabla f\left(\sum_{1 \leq i < j \leq n} x_{i,j} \tilde{e}_{i,j}\right)\right\rangle \\ &= \frac{1}{2} \sum_{1 \leq i < j \leq n} \mu_{i,j} x_{i,j}^2 \langle \tilde{e}_{i,j}, \tilde{e}_{i,j}\rangle = \sum_{1 \leq i < j \leq n} \mu_{i,j} x_{i,j}^2, \end{aligned}$$

where $\mu_{i,j} = \left[(\lambda_i^2 + \lambda_j^2) - \frac{4\lambda_i^2 \lambda_j^2}{\lambda_i^2 + \lambda_j^2 + 2c}\right]$. In particular,

$$f(X) \leq \max_{1 \leq i < j \leq n} \mu_{i,j} \sum_{1 \leq i < j \leq n} x_{i,j}^2 = \frac{1}{2} \max_{1 \leq i < j \leq n} \mu_{i,j} \|X\|^2.$$

(4) Write $g(s,t) = s + t - 4st/(s + t + 2c)$ for $s \geq t \geq 0$. Its derivatives satisfy

$$\partial_s g = 1 - \frac{4t(t + 2c)}{(s + t + 2c)^2} \geq 0,$$

and

$$\partial_t^2 g = \frac{8s(s + 2c)}{(s + t + 2c)^3} \geq 0.$$

Increasing the larger squared singular value to λ_1^2 therefore increases the bound. Convexity in the other value places its maximum on $[0, \lambda_2^2]$ at an endpoint. The endpoints give $g(\lambda_1^2, 0) = \lambda_1^2$ and $g(\lambda_1^2, \lambda_2^2) = \mu_{1,2}$, proving the claim.

□

B.2 Alignment on the Real Special Orthogonal Group

For $P \in SO(m)$, write

$$\|P\|_g = \min\{\|X\|_F : X \in \mathfrak{so}(m),\ e^X = P\}$$

for its geodesic distance from the identity.

Theorem B.2 (Alignment for real matrices). *Let $A \in \mathbb{R}^{n \times d}$ with $n > d \geq 2$ have top two singular values $\lambda_1 \geq \lambda_2$. For $c > 0$, define*

$$\mu_{1,2}(A,c) = \lambda_1^2 + \lambda_2^2 - \frac{4\lambda_1^2 \lambda_2^2}{\lambda_1^2 + \lambda_2^2 + 2c}.$$

For every $P \in SO(n)$,

$$\begin{aligned} &\min_{Q \in SO(d)} \left(\|PA - AQ\|_F^2 + c\|Q\|_g^2\right) \\ &\qquad \leq \frac{1}{2} \max\{\mu_{1,2}(A,c), \lambda_1^2\} \|P\|_g^2. \end{aligned}$$

Proof. Choose $X \in \mathfrak{so}(n)$ with $e^X = P$ and $\|X\|_F = \|P\|_g$. Let $Y \in \mathfrak{so}(d)$ minimize $\|XA - AY\|_F^2 + c\|Y\|_F^2$. For $C(t) = e^{tX} A e^{-tY}$,

$$C'(t) = e^{tX}(XA - AY)e^{-tY}, \qquad \|C'(t)\|_F = \|XA - AY\|_F.$$

Integration and orthogonal invariance give

$$\|PA - Ae^Y\|_F = \|C(1) - C(0)\|_F \leq \|XA - AY\|_F.$$

The choice $Q = e^Y$ has $\|Q\|_g \le \|Y\|_F$. Applying Theorem B.1 proves the claim. Taking $c \downarrow 0$ gives the corresponding unpenalized bound. □

B.3 Gaussian Matrix Mechanism (Theorem 5.1)

Proof. Let $z := f(x) + \mathcal{N}(0, \sigma^2\Sigma)$. For any $x, x' \in \mathcal{X}$, we have

$$p_{z|x}(z) \sim \mathcal{N}(f(x), \sigma^2\Sigma), \quad p_{z|x'}(z) \sim \mathcal{N}(f(x'), \sigma^2\Sigma).$$

Hence we obtain

$$\begin{aligned} D_\alpha(p_{z|x} \,\|\, p_{z|x'}) &= \frac{\alpha}{2}(f(x) - f(x'))^\top \sigma^{-2}\Sigma^{-1}(f(x) - f(x')) \\ &= \frac{\alpha}{2\sigma^2}\|\Sigma^{-1/2}(f(x) - f(x'))\|_2^2 \\ &\le \frac{\alpha L_2(f)^2}{2\sigma^2}\, d(x, x')^2 = \epsilon\, d(x, x')^2. \end{aligned}$$

□

B.4 Exponential Token Mechanism (Theorem 5.2)

Proof. By S_n-invariance of the Cayley metric d, the map $z \mapsto \sigma z$ is an isometry of the output space for every $\sigma \in S_n$. Hence, whenever x' lies at finite d-distance from x, there exists $\sigma \in S_n$ with $x' = \sigma x$, and

$$\begin{aligned} \mathcal{Z}(x') &= \sum_{z \in X_\ell} \exp\big(-\varepsilon_{\text{on}}\, d(x', z)\big) \\ &= \sum_{z \in X_\ell} \exp\big(-\varepsilon_{\text{on}}\, d(\sigma x, \sigma z)\big) = \mathcal{Z}(x). \end{aligned}$$

The equal normalizing constants cancel in the likelihood ratio:

$$\begin{aligned} \frac{\Pr[M_{\text{exp}}(x) = z]}{\Pr[M_{\text{exp}}(x') = z]} &= \exp\big(\varepsilon_{\text{on}}(d(x', z) - d(x, z))\big) \\ &\le \exp\big(\varepsilon_{\text{on}}\, d(x, x')\big) \end{aligned}$$

by the triangle inequality on d. Consequently, for every $\alpha > 1$,

$$\begin{aligned} &D_\alpha(M_{\text{exp}}(x) \| M_{\text{exp}}(x')) \\ &= \frac{1}{\alpha - 1} \log \sum_z \left(\frac{p(z|x)}{p(z|x')}\right)^{\alpha - 1} p(z \mid x) \\ &\le \varepsilon_{\text{on}}\, d(x, x'). \end{aligned}$$

Exact sampling for token sequences.

Lemma B.2 (Permutation sampling for the exponential mechanism). *Let $t = e^{\varepsilon_{on}}$ and $(t)_m = \prod_{j=0}^{m-1}(t + j)$, with $(t)_0 = 1$. A permutation $\sigma \in S_n$ sampled with probability*

$$\Pr(\sigma) = \frac{t^{c(\sigma)}}{(t)_n},$$

where $c(\sigma)$ counts its cycles, gives $\sigma x \sim M_{exp}(x)$. For a request with k distinct tokens, the normalizing constant for the sequence distribution is

$$\mathcal{Z} = \frac{(t)_n}{t^k (t)_{n-k}}.$$

Proof. An exact permutation sampler inserts vocabulary IDs in order. When inserting ID j, it starts a new cycle with probability $t/(t + j - 1)$. Each of the $j - 1$ positions after an existing ID has probability $1/(t + j - 1)$. Every final permutation has a unique reverse deletion history, so its probability is $t^{c(\sigma)}/(t)_n$.

For an output z obtained by consistently renaming the token IDs of x, the mapping $x \mapsto z$ specifies k directed edges on the n vocabulary IDs. They form disjoint closed cycles and open chains, including isolated IDs. If c_0 cycles are already closed, there are $n - k$ open chains. Completing the mapping is equivalent to permuting these chains as blocks. Hence

$$\sum_{\sigma : \sigma x = z} t^{c(\sigma)} = t^{c_0}(t)_{n-k}.$$

The maximum possible number of cycles in a completion is $c_0 + n - k$, giving $d(x, z) = k - c_0$. Thus

$$\Pr(\sigma x = z) = \frac{t^{c_0}(t)_{n-k}}{(t)_n} = \frac{e^{-\varepsilon_{\text{on}} d(x,z)}}{\mathcal{Z}}.$$

This proves the claimed sequence law. □

Corollary B.1 (Online bound with request-independent offline state). *Let the offline state S have any distribution independent of the request. Suppose it determines a permutation π_S and checkpoint C_S. With an independent draw from $M_{exp}(x)$, the release $(C_S, \pi_S(M_{exp}(x)))$ satisfies $(\alpha, \varepsilon_{on}, d)$-RmDP.*

Proof. Couple the same S in both comparisons. Conditional on $S = s$, the checkpoint agrees and the common permutation π_s preserves the divergence between the two online output distributions. Each conditional divergence is at most $\varepsilon_{\text{on}} d(x, x')$ by Theorem 5.2. Joint convexity of $\exp((\alpha - 1)D_\alpha)$ transfers this bound to the released mixture. □

B.5 Submechanism Composition for a Shared Key

Theorem B.3 (Shared-key variational bound). *Let $\theta, \xi \in \mathbb{R}^{n \times d}$ and $\alpha > 1$. Let $H \in GL_d(\mathbb{R})$ have a distribution invariant under left multiplication by $SO(d)$. Let $E, F \in \mathbb{R}^{n \times d}$ have independent Gaussian entries of variances $\sigma^2, \tau^2 > 0$, independently of each other and of H. Conditional on H, assume the randomness of $\mathcal{M}_1(H)$ is independent of E, F. Assume that, for every $h \in GL_d(\mathbb{R})$ and $Q \in SO(d)$,*

$$D_\alpha(\mathcal{M}_1(h) \| \mathcal{M}_1(Qh)) \le \epsilon_1 \|Q\|_g^2$$

for a common $\epsilon_1 \ge 0$. Define, for $g \in O(n)$,

$$\mathcal{M}(g) = \big(g(\theta + E)H,\ \mathcal{M}_1(H),\ g(\xi + F)H^{-T}\big).$$

For $A = \theta/\sigma + i\xi/\tau$ and $\Delta g = g_2^{-1} g_1$,

$$\begin{aligned} &D_\alpha(\mathcal{M}(g_1) \| \mathcal{M}(g_2)) \\ &\quad \le \inf_{Q \in SO(d)} \left\{ \frac{\alpha}{2} \|\Delta g A - AQ\|_F^2 + \epsilon_1 \|Q\|_g^2 \right\}. \end{aligned}$$

Proof. For any $Q \in SO(d)$, couple $H_2 = QH_1$. The two key marginals coincide. Conditional on $H_1 = h$, cancel the common right factors h and h^{-T} from the first and third outputs, respectively. Here $(Qh)^{-T} = Qh^{-T}$. The two real Gaussian divergences sum to $\alpha\|\Delta g A - AQ\|_F^2/2$. Conditional independence and the uniform child bound add at most $\epsilon_1\|Q\|_g^2$. Joint convexity of $\exp((\alpha - 1)D_\alpha)$ transfers this bound from the coupled conditional distributions to the released mixture. Taking the infimum over Q proves the claim. □

This variational result describes a square shared key with the stated conditional child bound. The complete checkpoint with rectangular encoding and decoding matrices is covered by the common release map in the following proof.

B.6 Offline Private Budget (Proof of Theorem 5.3)

Proof. Consider the real Gaussian release

$$Z_\pi = \left(\Pi_\pi(W_{\text{embed}} + E_e), (W_{\text{head}} + E_h)\Pi_\pi^\top\right).$$

Here E_e, E_h denote the interface noise with independent centered Gaussian entries of standard deviations s_e, s_h. The offline construction produces $M_{\text{off}}(\pi) = F(Z_\pi, R)$, where R collects the remaining offline randomness and F is common to all π for the fixed plaintext checkpoint and public release parameters. The two noise arrays are isotropic on their respective real matrix spaces, and vocabulary permutations preserve their distributions. Adding the independent variable R preserves Rényi divergence, and applying F gives

$$D_\alpha(M_{\text{off}}(\pi)\|M_{\text{off}}(\pi')) \le D_\alpha(Z_\pi\|Z_{\pi'}).$$

Write $\Delta = \Pi_\pi - \Pi_{\pi'}$ and $m = d(\pi, \pi')$. The equal-covariance Gaussian formula yields

$$\begin{aligned} D_\alpha(Z_\pi\|Z_{\pi'}) &= \frac{\alpha}{2}\left(\frac{\|\Delta W_{\text{embed}}\|_F^2}{s_e^2} + \frac{\|\Delta W_{\text{head}}^\top\|_F^2}{s_h^2}\right) \\ &= \frac{\alpha}{2}\|\Delta A_\partial\|_F^2 \\ &\le \frac{\alpha}{2}\lambda_1^2\|\Delta\|_F^2. \end{aligned}$$

Here A_∂ compactly records two real squared norms. The last inequality follows from the spectral norm of A_∂. A product of m transpositions moves at most $2m$ vocabulary IDs. Each moved ID contributes two to the squared Frobenius distance between permutation matrices, so

$$\|\Delta\|_F^2 \le 4m, \qquad D_\alpha(M_{\text{off}}(\pi)\|M_{\text{off}}(\pi')) \le 2\alpha\lambda_1^2 m.$$

For $m \ge 1$, the displayed parameter of Theorem 5.3 satisfies

$$2\alpha\lambda_1^2 m \le \frac{\alpha\omega^2}{2}\lambda_1^2 m^2 \le \varepsilon_{\text{off}} m^2.$$

For $m = 0$, the two output distributions coincide and their divergence is zero. This proves the stated bound for every $\alpha > 1$. The map F may transform the interfaces with distinct rectangular encoding and decoding matrices and generate intermediate weights from R. □

C Architecture Details

This appendix develops the module constructions in Section 4. We specify the operator dimensions, token permutations, expanded keys, and noise multipliers below. The algebraic identities below compare compatible transformations of the same underlying weights. Additive weight noise, positional-frequency mixing, scalar normalization calibration, and finite-precision evaluation contribute their corresponding computation errors.

C.1 Operators and Dimensions

Let d be the plaintext residual width, $d' = d + 2h$ the expanded residual width, n the vocabulary size, and d_{a} the width of an attention head. Row-vector notation gives $W_{\text{embed}} \in \mathbb{R}^{n\times d}$ and $W_{\text{head}} \in \mathbb{R}^{d\times n}$. A token index selects an embedding row. The output head maps a residual vector to n vocabulary logits. For a gated FFN with intermediate width m, $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d\times m}$ and $W_{\text{down}} \in \mathbb{R}^{m\times d}$, with

$$F(x) = \left(\text{SiLU}(xW_{\text{gate}}) \odot (xW_{\text{up}})\right)W_{\text{down}}.$$

MoE experts have this same local form and are selected by a router.

Let m_q be the number of query heads and m_{kv} the number of shared key/value heads. If $\eta(i)$ maps query head i to its key/value head, attention takes the form

$$A(x) = \sum_{i=1}^{m_q} \text{softmax}\left(\frac{\mathcal{R}(Q_i)\mathcal{R}(K_{\eta(i)})^\top}{\sqrt{d_{\text{a}}}}\right) V_{\eta(i)} W_o^{(i)},$$

where $Q_i = xW_q^{(i)}$, $K_j = xW_k^{(j)}$, $V_j = xW_v^{(j)}$, and $\mathcal{R}$ applies the position-dependent RoPE rotations. MHA has $m_{kv} = m_q$, MQA has $m_{kv} = 1$, and GQA shares a key/value head across a group of query heads [1, 54].

C.2 Expanded Key Matrices

Choose an even expansion parameter h. The initialization draws U uniformly from $O(d)$, draws independent entries $V_{ij} \sim \mathcal{N}(0, 1/d)$, and forms the base matrix

$$B = U + \lambda V.$$

The low-rank expansion blocks are

$$E_{\text{lr}} = E_1 E_2 \in \mathbb{R}^{d\times h}, \qquad F = F_1 F_2 \in \mathbb{R}^{h\times d},$$

with $E_1 \in \mathbb{R}^{d\times h/2}$, $E_2 \in \mathbb{R}^{h/2\times h}$, $F_1 \in \mathbb{R}^{h\times h/2}$, and $F_2 \in \mathbb{R}^{h/2\times d}$. The entries of these factors are independently drawn from $\mathcal{N}(0, 1/d)$. A uniformly sampled orthogonal matrix $R \in O(d')$ mixes the expanded coordinates.

To specify the nullspace orientations, let the columns of N_F span $\ker(F^\top)$ and the columns of N_E span $\ker(E_{\text{lr}})$. For free coordinate matrices A_C, A_D of matching dimensions, set

$$C = A_C N_F^\top, \qquad D = N_E A_D.$$

Thus $C \in \mathbb{R}^{d\times h}$, $D \in \mathbb{R}^{h\times d}$, $CF = 0$, and $E_{\text{lr}}D = 0$. The encoding matrix and right inverse are

$$\widehat{P} = [B\ C\ E_{\text{lr}}]R, \qquad \widehat{Q} = R^\top \begin{pmatrix} B^{-1} \\ F \\ D \end{pmatrix}.$$

Their product is

$$\widehat{P}\widehat{Q} = BB^{-1} + CF + E_{\text{lr}}D = I_d.$$

The dimensions are $\widehat{P} \in \mathbb{R}^{d\times d'}$ and $\widehat{Q} \in \mathbb{R}^{d'\times d}$. They provide an injective encoding of plaintext states and a decoder on the expanded space.

The common B, E_{lr}, F, R allow different choices of C and D while retaining cross-compatibility. For any such pair C_i, D_j,

$$\widehat{P}_i\widehat{Q}_j = I_d.$$

These identities permit distinct weight transformations around a shared residual boundary. Residual summation additionally requires the branches to use a common output encoding. The parameter

λ changes the conditioning of B, and h determines the expansion size. The mathematical inverse exists almost surely under the stated continuous sampling law. Its numerical accuracy depends on the realized condition number and arithmetic precision.

C.3 Token Interface and Autoregressive Boundary

Define the permutation matrix by $(\Pi_\pi)_{\pi(i),i} = 1$. The row at transformed token ID $\pi(i)$ in $\Pi_\pi W^\star_{\text{embed}}\widehat{P}$ is $W^\star_{\text{embed}}[i,:]\widehat{P}$. Similarly, decoded residual state x has transformed output logits

$$(x\widehat{P})(\widehat{Q}W^\star_{\text{head}}\Pi_\pi^\top) = xW^\star_{\text{head}}\Pi_\pi^\top.$$

Vocabulary scores are therefore relabeled by the same permutation as the input IDs. Applying π^{-1} to a generated ID restores the corresponding plaintext ID. The new token can be appended to the history with this same correspondence at every autoregressive step.

These identities describe the token-ID boundary. A string-based exchange must preserve the mapped IDs through serialization and tokenization in both directions. The exponential mechanism samples the request before the secret permutation, as specified in Algorithm 2.

C.4 Attention Transformations and RoPE

Query and key orientation. Use a block-diagonal rotation R, a block-diagonal scale H with a nonzero scalar on each planar pair, and a pair-permutation matrix S. Define

$$T_q = RHS, \qquad T_k = RH^{-1}S.$$

Since R is orthogonal and H is symmetric,

$$T_qT_k^\top = RHSS^\top H^{-1}R^\top = I.$$

The query and key use the same orientation of S in these expressions. For an input encoding $x\widehat{P}$, the residual-side requirements are $\widehat{P}\widehat{Q}_q = \widehat{P}\widehat{Q}_k = I_d$. They give the transformed queries and keys qT_q and kT_k before positional rotation.

Position-dependent identity. For position t, write

$$R_t = \operatorname{diag}\big(R(t\omega_1), \ldots, R(t\omega_{d_a/2})\big),$$

where $R(\cdot)$ is the two-dimensional rotation matrix and the ω_j are the model's rotary frequencies [58]. An original query at position s and key at position t have score $qR_{s-t}k^\top$. The transformed score is

$$qRHSR_{s-t}S^\top H^{-1}R^\top k^\top.$$

It equals the original score when S commutes with R_{s-t}. The blockwise rotations and scalar pairwise scales already commute with the original positional rotations. Permuting pairs with the same frequency preserves this condition. Reordering the rotary frequencies together with the coordinate pairs also preserves it: using $R'_t = S^\top R_t S$ gives $T_qR'_{s-t}T_k^\top = R_{s-t}$.

Frequency mixing and approximation. Mixing RoPE pairs within bounded windows introduces an attention-score approximation error. When frequencies remain in their original positions, the absolute score error is bounded by

$$\begin{aligned}&\left|qT_qR_{s-t}T_k^\top k^\top - qR_{s-t}k^\top\right| \\ &\quad\le \|q\|_2\|k\|_2\ \operatorname{cond}_2(H)\left\|SR_{s-t}S^\top - R_{s-t}\right\|_2.\end{aligned}$$

If $p(j)$ is the reassignment of pair frequencies induced by S, the final norm equals

$$\max_j 2\left|\sin\left(\frac{(s-t)(\omega_{p(j)} - \omega_j)}{2}\right)\right|.$$

It is bounded by $\min\{2, |s-t|\max_j|\omega_{p(j)} - \omega_j|\}$. The scaled attention score includes the additional factor $1/\sqrt{d_a}$. Thus this error depends on the window size, frequency differences, distance between token positions, and conditioning of the scaling matrix.

Value and output path. For an invertible value-space matrix U_v, define $\widetilde{W}_v = \widehat{Q}_vW_vU_v$ and $\widetilde{W}_o = U_v^{-1}W_o\widehat{P}_o$. Then

$$(x\widehat{P})\widetilde{W}_v\widetilde{W}_o = xW_vW_o\widehat{P}_o$$

whenever $\widehat{P}\widehat{Q}_v = I_d$. This is the value/output cancellation. Equality of the full attention output additionally uses equality of the attention probabilities. When weights include additive noise, the identity applies to the same noisy weights on both sides, and their difference from the original weights contributes a local computation error.

C.5 Shared Heads and Latent Attention

In GQA and MQA, each key/value head has one shared key transformation and one shared value transformation. Every query head in its group uses the compatible query-side factor, and its output projection uses the inverse of the shared value factor. Let σ permute key/value groups and let a within-group permutation reorder query heads. The corresponding output blocks are reordered with those query heads. This changes the order of summation in the attention expression and preserves the group-sharing relation. Independently selecting incompatible factors for two queries that share a key would break that relation.

MLA [13] factors a content projection through a latent state. For a factorization xAB, an invertible latent change T gives

$$x(AT)(T^{-1}B) = xAB.$$

Every projection that consumes this shared latent state uses the same inverse change. The decoupled rotary path has its own positional compatibility conditions from Section C.4. This factor identity states the required cancellation independently of a model's particular tensor names. A concrete model transformation records the content, positional, and latent factors separately.

C.6 FFN and Expert Routing

Let Z_f permute the FFN intermediate coordinates and let H_f be an invertible diagonal matrix. Use

$$\widetilde{W}_g = \widehat{Q}_gW_gZ_f, \qquad \widetilde{W}_u = \widehat{Q}_uW_uH_fZ_f,$$

and

$$\widetilde{W}_d = Z_f^{-1}H_f^{-1}W_d\widehat{P}_d.$$

With $\widehat{P}\widehat{Q}_g = \widehat{P}\widehat{Q}_u = I_d$ and $\widetilde{x} = x\widehat{P}$, elementwise activation and the Hadamard product give

$$\begin{aligned}&\operatorname{SiLU}(\widetilde{x}\widetilde{W}_g)\odot(\widetilde{x}\widetilde{W}_u) \\ &\quad= \big(\operatorname{SiLU}(xW_g)\odot(xW_u)\big)H_fZ_f.\end{aligned}$$

Multiplication by $\widetilde{W}_d$ then gives $F(x)\widehat{P}_d$. The scale is carried by the linear up branch, while the activated branch uses a permutation.

The same derivation applies to other elementwise activations under this placement of the scale.

For a router logit row $r = xW_{\text{router}}$ and expert permutation S_e, the transformed logits are rS_e when the input encoding is canceled. Softmax obeys $\text{softmax}(rS_e) = \text{softmax}(r)S_e$. Top-$k$ selects the same experts under the matching relabeling when ties are handled consistently. Reordering the expert functions and their weights by the same permutation preserves their weighted sum.

Let $\delta r = x(W'_{\text{router}} + E_{\text{router}} - W_{\text{router}})$ include both normalization and noise. The original selected set is preserved when the gap between the kth and $(k+1)$th logits exceeds $2\|\delta r\|_\infty$. The coordinate permutation is exact for the perturbed logits as well. This separates expert relabeling from the effect of weight perturbation on the chosen set.

C.7 RMSNorm and Residual Boundaries

For width d and stabilizer ϵ_d, define

$$r_d(x) = \sqrt{\|x\|_2^2/d + \epsilon_d}.$$

RMSNorm with learned diagonal gain D_g is $xD_g/r_d(x)$ [75]. Its gain can be fused into an adjacent projection W as D_gW, because $(xD_g/r_d(x))W = x(D_gW)/r_d(x)$.

With expansion, a scalar gain κ gives the normalized row

$$\kappa x\widehat{P}/r_{d'}(x\widehat{P}).$$

The exact scalar correction at a particular activation is

$$\kappa(x) = \frac{r_{d'}(x\widehat{P})}{r_d(x)}.$$

With zero stabilizers and nonzero x, this becomes

$$\kappa(x) = \sqrt{\frac{d}{d'}}\,\frac{\|x\widehat{P}\|_2}{\|x\|_2}.$$

A constant estimated from calibration data approximates this ratio. The relevant ratio includes the width normalization and both stabilizers. An orthogonal transformation at unchanged width preserves the RMS value exactly. A general rectangular encoding requires the scalar approximation or another specified treatment of the norm.

For a residual addition $x + u$, a common output encoding gives $(x+u)\widehat{P} = x\widehat{P} + u\widehat{P}$. Both branches must use that same realized matrix at the addition. Their subsequent projections may use different right inverses from the compatible family above. These requirements connect the local weight identities to the residual composition proof in Appendix A.

C.8 KV Cache and Serving

The transformed model retains the sequence of attention, FFN, normalization, and residual operators. The actual tensor dimensions determine the KV-cache allocation and the projections read by the serving engine. Paged attention indexes cached blocks by sequence and position [33]. Each cache entry must correspond to the transformed key/value representation expected by its query group. Continuous batching and streaming preserve these correspondences within one tenant instance. The expansion experiment in Appendix D measures the resulting latency for the reported configuration.

Table 7: Reported offline accounting values at $\varepsilon_{\text{on}} = 20$, $k = 100$, and $\alpha = 2$. For every row, $\varepsilon_{\text{full}} \approx \varepsilon_{\text{on}}$ at the reported precision.

Model	ε_{off}
Qwen2.5-14B-Instruct	3.47×10^9
Llama3-8B (σ_e=0.5)	5.8610×10^9
DeepSeek-R1-Distill-14B	3.0129×10^9
DeepSeek-R1-Distill-32B	3.9703×10^9
Qwen3-14B	2.6620×10^9
Qwen3-32B	3.9824×10^9
Qwen3-MoE-30B-A3B	3.0401×10^9
DeepSeek-V3.1-671B	2.2339×10^9

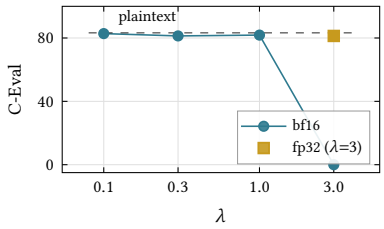


Figure 5: Effect of numerical precision on the key-matrix conditioning parameter λ at $(\sigma_e, \sigma_h) = (1.0, 0.2)$ on Qwen2.5-14B-Instruct. At $\lambda = 3.0$, bf16 accuracy degrades while fp32 retains C-Eval 81.30.

D Additional Experiments

Table 7 records the offline accounting values for the evaluated models. Every row uses $\varepsilon_{\text{on}} = 20$, $k = 100$, and $\alpha = 2$. Noise multipliers are $(\sigma_e, \sigma_h) = (1.0, 0.2)$ except Llama3-8B at $\sigma_e = 0.5$. Middle-layer Gaussian noise uses $\rho = 0.05$ in the standard-deviation-scaled rule of Section 6.1. In every row, Theorem 5.4 yields $\varepsilon_{\text{full}} \approx \varepsilon_{\text{on}} = 20$ at the reported precision.

Figure 5 varies $\lambda \in \{0.1, 0.3, 1.0, 3.0\}$ under bf16 and fp32 on Qwen2.5-14B-Instruct, showing bf16 collapses at $\lambda = 3.0$ while fp32 holds at C-Eval 81.30.

Across the ten sweep entries, including the repeated default configuration, the Spearman rank correlation between the reported offline accounting values and TTRSR is 0.89. The evaluation covers 10 points: 5 varying σ_e at σ_h=0.2 and 5 varying σ_h at σ_e=1.0, anchored on Qwen2.5-14B-Instruct.

Influence of Expansion Size h. The expansion parameter h determines how much the hidden dimension increases. Figure 6 varies h from 128 to 512 on R1-Distill-14B at concurrency 4. TPOT growth stays below 10% even at $h = 512$, as serving engines parallelize the expanded matrix work and absorb it in the existing tensor-parallel layout.

E Open Science

The source code and artifacts supporting the evaluation of TWIST are publicly available at https://doi.org/10.5281/zenodo.22709384. The artifact includes the implementation, scripts, configurations, and documentation needed to reproduce the main experiments reported in this paper.

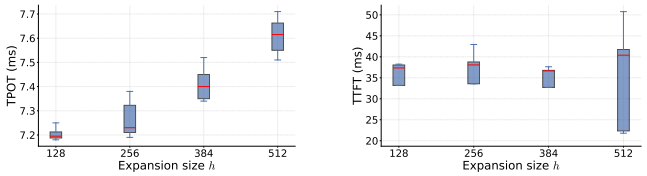


Figure 6: Online inference latency on R1-Distill-14B at concurrency 4 as the intermediate expansion size h ranges from 128 to 512. (a) TPOT. (b) TTFT.

F Ethical Considerations

TWIST is a defensive privacy mechanism for protecting tenant data during outsourced LLM inference. Our evaluation uses public datasets and does not involve human subjects or private user data. The attacks considered in this work are used only to evaluate privacy leakage under the threat model in Section 2.2. We do not conduct attacks against deployed third-party services or systems.

G Proof of the Offline-Online Joint Mechanism

Proof. Let $g = \pi$ and $z := (z_1, z_2) = \big(gM_{\text{exp}}(x), M_{\text{off}}(g)\big)$. Below, $\mathcal{M}$ denotes the joint mechanism, and $\mathcal{M}_1, \mathcal{M}_2$ denote its first and second components.

Since $p_{\mathcal{M}}(z|x,g) = p_{\mathcal{M}_1}(z_1|g,x)\cdot p_{\mathcal{M}_2}(z_2|g)$, we have

$$\begin{aligned} &D_\alpha(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')) \\ &= D_\alpha(p_{\mathcal{M}_1}(z_1|g,x) \parallel p_{\mathcal{M}_1}(z_1|g',x')) \\ &\quad + D_\alpha(p_{\mathcal{M}_2}(z_2|g) \parallel p_{\mathcal{M}_2}(z_2|g')). \end{aligned}$$

By assumption, $D_\alpha(p_{\mathcal{M}_2}(z_2|g) \parallel p_{\mathcal{M}_2}(z_2|g')) \le \varepsilon_{\text{off}} d^2(g,g')$. Vocabulary relabeling preserves the distance and normalizing constant of M_{exp}, giving equivariance. From Theorem 5.2 and this property, we have

$$\begin{aligned} &D_\alpha(p_{\mathcal{M}_1}(z_1|g,x) \parallel p_{\mathcal{M}_1}(z_1|g',x')) \\ &\le \varepsilon_{\text{on}} d(gx, g'x') = \varepsilon_{\text{on}} d(x, \Delta g x'), \end{aligned}$$

where $\Delta g = g^{-1}g'$. Therefore

$$\begin{aligned} &D_\alpha(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')) \\ &\le \varepsilon_{\text{on}} d(x, \Delta g x') + \varepsilon_{\text{off}} d^2(g,g'), \end{aligned}$$

for any $\Delta g \in S_n$.

Let $d(x,x') = r$. Then there exists $h \in S_n$ such that $x' = hx$ and $d(h,1) = r$, and there is a corresponding shortest transposition chain $1 = h_0 \to h_1 \to \cdots \to h_r = h$. Thus, for each $i = 0, 1, \ldots, r$, define $\Delta g_i := h_{r-i}h^{-1}$ and $g' = g\Delta g_i$. Then

$$\begin{aligned} d(x, \Delta g_i x') &= d(x, h_{r-i}h^{-1}hx) \\ &= d(x, h_{r-i}x) = r - i, \\ d(g,g') &= d(\Delta g_i, 1) = d(h_{r-i}h^{-1}, 1) = i. \end{aligned}$$

Hence

$$\begin{aligned} &D_\alpha(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')) \\ &\le \varepsilon_{\text{on}}(r-i) + \varepsilon_{\text{off}} i^2 =: f_r(i). \end{aligned}$$

The function $f_r(i)$ is quadratic in i. Minimizing over i gives $\min_{i=0,\ldots,r} f_r(i) \le \phi(r)$, where

$$\phi(r) = \begin{cases} \varepsilon_{\text{off}} r^2 & r \le \left\lfloor \dfrac{\varepsilon_{\text{on}}}{2\varepsilon_{\text{off}}} \right\rfloor, \\ f_r\left(\left\lfloor \dfrac{\varepsilon_{\text{on}}}{2\varepsilon_{\text{off}}} \right\rfloor\right) & \text{otherwise.} \end{cases}$$

In the second regime, $\phi(r) \approx \varepsilon_{\text{on}} r - \varepsilon_{\text{on}}^2/(4\varepsilon_{\text{off}})$. For $x, x' \in \mathbb{Z}_n^\ell(k)$, the distance ranges over $r \in [0, k-1]$. Hence, the two regimes give $\epsilon = \varepsilon_{\text{full}}$ as stated in the theorem. Therefore the minimizing choice of i gives

$$\begin{aligned} &D_\alpha(p_{\mathcal{M}}(z|x,g) \parallel p_{\mathcal{M}}(z|x',g')) \\ &\le \epsilon d(x,x'). \end{aligned}$$

Take g uniform and $g' = g\Delta g_i$, which also has a uniform marginal. For $\alpha > 1$, $\exp((\alpha-1)D_\alpha(P \parallel Q))$ is jointly convex in (P,Q). Taking expectations over g, g', we get

$$e^{(\alpha-1)D_\alpha(M^\partial_{\text{full}}(x) \| M^\partial_{\text{full}}(x'))} \le e^{(\alpha-1)\epsilon d(x,x')},$$

which yields $D_\alpha(M^\partial_{\text{full}}(x) \parallel M^\partial_{\text{full}}(x')) \le \epsilon d(x,x')$. □

H Notation

The following notation is shared by the conference paper and this extended version.

Table 8: Notation used throughout the paper.

Symbol	Meaning
n, ℓ, k	Vocabulary size, request length, and bound on distinct request tokens.
d, h	Original hidden width and expansion size. The expanded width is $d + 2h$.
$\theta, \widetilde{\theta}$	Plaintext and transformed model weights.
π, Π_π	Token permutation and its permutation matrix.
ϖ	The circle constant.
G	Family of module-specific weight transformations.
E	Offline additive noise tensors.
$\widehat{P}, \widehat{Q}$	Expanded key matrix and compatible right inverse, with $\widehat{P}\widehat{Q} = I_d$.
λ	Key-matrix conditioning parameter.
$\lambda_i(A)$	Singular values of matrix A, in descending order.
σ_e, σ_h	Dimensionless embedding and head noise multipliers.
$s_e, s_h, s_{t,j}$	Absolute Gaussian standard deviations for token-interface and middle-layer weights.
ρ	Middle-layer noise multiplier in $E = \rho\,\text{std}(W)\Xi$.
α	Rényi divergence order.
$\varepsilon_{\text{on}}, \varepsilon_{\text{off}}, \varepsilon_{\text{full}}$	Online, offline, and composed RmDP parameters.
$M_{\text{exp}}, M_{\text{off}}, M^\partial_{\text{full}}$	Online token mechanism, offline release, and their joint mechanism.
E_{lr}, R	Key-expansion block and orthogonal mixer.
β, γ	Maximum attention-block window size and window-sampling parameter, respectively.

The released configuration uses `embed_noise_factor`, `head_noise_factor`, `matrix_coefficient`, and `flex_hidden_size` for σ_e, σ_h, λ, and h, respectively. Its configuration field `beta` denotes the middle-layer noise multiplier ρ. The manuscript uses β, γ for attention block permutation.

I Evaluation Details

Models. We evaluate TWIST on one primary model and seven additional open-weight models, covering dense and MoE architectures.

- **Dense models.** We use Qwen2.5-14B-Instruct [51] as the shared open-weight base model for head-to-head baseline comparison, ablations, and privacy-utility analysis. We further select 5 models (Llama3-8B [22], Qwen3-14B/32B [72], and R1-Distill-14B/32B [12]) to represent the 8B–32B range typical for dedicated deployments.
- **MoE architectures.** To validate our MoE-specific transformation (Section 4.7), we further include Qwen3-MoE-30B-A3B and the 671B DeepSeek-V3.1-Terminus [11].

Table 4 reports the seven additional models and Table 5 reports timing for four of them, while Table 2–Table 3 and Section 7.4 use the primary model.

Datasets. We evaluate both utility and privacy:

- **Utility.** 5 benchmarks span distinct capabilities: *knowledge* (MMLU [26], C-Eval [28]), *coding* (HumanEval [7]), *reasoning* (PIQA [4]), and *instruction following* (IFEval [79]). Baseline tables also report SST-2 [56].
- **Privacy.** PUPA [35] supports recovery evaluation for individual prompts. CCI3 [69], Huatuo26M-Lite [70], and MedDialog [8] are used for frequency analysis over accumulated requests (Section 7.6).

Baselines. We select 4 data-only baselines covering token perturbation, Gaussian noise, and learned transformations:

- **Token-level.** SANTEXT [73] and RANTEXT [62] perform discrete substitution or perturbation on raw input tokens.
- **Hidden-state perturbation.** Our DP-Forward [16] adaptation adds calibrated Gaussian noise after the first decoder layer. The client runs the embedding and first decoder layer and sends the perturbed hidden states to the provider.
- **Embedding-level (Learned).** Stained Glass (SGT) [52] replaces analytical noise with a learned transformation to balance utility and privacy.

The comparison uses Qwen2.5-14B-Instruct as the shared base model. We do not include cryptographic or TEE-based approaches in the quantitative comparison because they require different computation protocols or hardware [25, 34, 59, 64]. Their computational overhead and hardware requirements differ from those of TWIST, which uses unmodified serving engines.

Task-specific fine-tuning. The SST-2 results use task-specific LoRA fine-tuning [27]. We apply LoRA to the attention weights with rank 64 and LoRA parameter $\alpha_{\text{LoRA}} = 64$, using a learning rate of 2×10^{-5} for 3 epochs, a sequence length of 128, and a batch size of 64. SANTEXT and RANTEXT perturb the training and validation data before fine-tuning and inference. Our DP-Forward adaptation injects noise during both fine-tuning and inference, whereas SGT and TWIST apply obfuscation only at inference. The generation benchmarks use no task-specific fine-tuning. The baseline privacy budgets are 3 for SANTEXT, 30 for RANTEXT, and 5 for DP-Forward. Our DP-Forward adaptation uses local DP with sensitivity 1 on unnormalized hidden states.

Privacy Metrics. We quantify the privacy leakage of TWIST using the metrics below. For all recovery metrics, lower values indicate stronger privacy.

- **TTRSR**: measures the proportion of original tokens successfully recovered by the attacker. Token recovery is a conservative metric: exact token identification implies complete exposure of the underlying discrete representation. We evaluate recovery using Vocabulary-Matching Attack (VMA) [61] and Invariant Attack (IA) [40].
- **Recovered-Text Cosine Similarity (CosSim)**: calculates the cosine similarity between the embedding of the recovered text and the original plaintext. We use CosSim to measure semantic similarity between the recovered and original text. We evaluate recovery using Internal State Attack (ISA) [15], Inversion Model Attack (IMA) [32], and nearest-neighbor (NN) attack.
- **PIIRSR**: our measure of the personally identifiable information recovery rate on the same inversion outputs.

Section 7.6 evaluates token-frequency leakage with Token Frequency Matching Attack (TFMA) and Substitution Deciphering Attack (SDA) [2]. TFMA is reported as TTRSR at top-10 and top-100, and SDA as a BLEU-4 score.

Hardware. The client runs on two Intel(R) Xeon(R) Platinum 8457C processors (96 cores) without a GPU. The provider uses a GPU cluster for LLM inference. We deploy the provider's inference service with vLLM 0.9.1 [33] and CUDA 12.4.

J Attack Constructions

The attack families in Section 7 target different provider-visible quantities. This appendix describes these attack methods and states the dimensions and assumptions behind their invariants. The numerical measurements remain those reported in Table 2–Table 6.

J.1 Vocabulary Matching

A basic vocabulary-matching problem observes

$$Y = P_r X P_c,$$

where $X \in \mathbb{R}^{n\times m}$ is public and P_r, P_c are unknown row and column permutations [61]. Let RowSort sort the entries within each row. Then

$$\mathrm{RowSort}(Y) = P_r\,\mathrm{RowSort}(X),$$

because sorting removes the column permutation while retaining the row order. Matching sorted rows yields candidates for the vocabulary permutation. Repeated or similar sorted rows make the match ambiguous. Additive noise changes the row signatures and turns exact equality into a distance-based matching problem.

Vocabulary matching uses combinations of embedding, output, and internal projection weights. Representative public quantities are $W_{\text{embed}}W_{\text{head}}$, $W_{\text{embed}}W_q(W_{\text{embed}}W_k)^\top$, $W_{\text{embed}}W_g$, $W_{\text{embed}}W_u$, W_dW_{head}, and $W_{\text{embed}}W_{\text{router}}$. Their dimensions differ: the first two are vocabulary-by-vocabulary matrices, while the gate and up combinations expose an intermediate feature axis. A matching invariant requires the remaining feature-space transformation to belong to a class removed by the chosen normalization or signature. An arbitrary two-sided linear transformation changes these signatures.

For multiple decoder layers, let $c_t(a)$ be the plaintext candidate assigned to observed token a by layer t. A voting score is

$$S(a,b) = \sum_t \mathbf{1}\{c_t(a) = b\}.$$

Selecting the largest score produces a per-token estimate. A one-to-one recovery of the entire permutation additionally imposes an assignment constraint across the token estimates. These are distinct outputs, so a reproduction records whether it scores per-token candidates or a global assignment.

J.2 Invariant Attacks

Gate-weighted means. Let $e \in \mathbb{R}^{1\times d}$ be a token embedding and $W_g \in \mathbb{R}^{d\times m}$ a gate projection. For a feature permutation Z_f,

$$\frac{1}{m}(eW_gZ_f)\mathbf{1} = \frac{1}{m}(eW_g)\mathbf{1}.$$

The mean is unchanged because $Z_f\mathbf{1} = \mathbf{1}$. Several gate layers produce a signature vector for the same vocabulary item. This supplies a token-matching statistic when the transformed embedding and gate projection reduce to a common feature permutation. Weight noise and incompatible feature transformations contribute changes to that signature.

Blockwise quadratic signatures. A dimensionally consistent form of the attention-block invariant uses a feature matrix $X \in \mathbb{R}^{n\times r}$ with full column rank and its token row $z \in \mathbb{R}^{1\times r}$. For an invertible block transformation $A \in \mathbb{R}^{r\times r}$,

$$(zA)\big((XA)^\top XA\big)^{-1}(zA)^\top = z(X^\top X)^{-1}z^\top.$$

Indeed,

$$(A^\top X^\top XA)^{-1} = A^{-1}(X^\top X)^{-1}A^{-\top},$$

and the factors cancel. A vocabulary permutation of X leaves its Gram matrix unchanged and reorders the resulting token signatures. The invariant equals a diagonal entry of the projector onto the projected block's column space. The feature row in the quadratic form has width r, matching the Gram matrix. Applying the formula to an unprojected width-d embedding would require a different matrix.

In attention, the attacker can form X from a public embedding matrix and a query or key block. Block and head permutations hide the correspondence between such projected subspaces. Noise perturbs the signatures themselves. A singular Gram matrix requires a specified treatment, such as regularization or a supported pseudoinverse identity. Regularization changes the exact invariant, and its parameter is part of the attack configuration.

J.3 Nearest-Neighbor and Learned Inversion

A nearest-neighbor attack compares an observed or reconstructed representation with the public embedding vocabulary and returns the nearest token under its chosen similarity measure. The comparison assumes that the two representations are expressed in compatible coordinates. The attack record therefore specifies the representation layer, any normalization, and the distance used for ranking.

An inversion-model attack learns a map from accessible transformed representations to plaintext tokens or embeddings [32]. The inversion model is based on Qwen2, with two decoder layers and eight attention heads. Its training pairs are generated from public text. The learned map's applicability depends on how the transformation used to generate training pairs relates to the tenant's transformation. The knowledge of the algorithm, the ability to sample new transformation keys, and access to the tenant's actual key are different attacker capabilities.

An internal-state attack instead optimizes a candidate input to match an observed runtime representation [15]. Writing $H_\theta(x)$ for the selected public-model representation and H_{obs} for the observed state, a typical optimization target is

$$\min_{x'} \mathcal{L}\big(H_\theta(x'), H_{\text{obs}}\big).$$

The target layer, representation coordinates, optimization domain, and loss determine the attack instance. Matching states produced by different weight systems additionally requires an alignment model. The reported attack evaluation measures recovery for its instantiated objectives.

J.4 Frequency Matching and Substitution Deciphering

A fixed token permutation preserves equality patterns and maps the frequency count of each input symbol to that of its permuted symbol. A frequency-matching attack compares counts or ranks from the observed token sequences with a plaintext prior corpus. The three priors in Table 6 are a public-corpus prior, a domain-aware medical-dialogue prior, and a distribution-aware prior from disjoint subsets of the same corpus. Their recovery rates measure

the effect of greater similarity between the prior and private text distributions.

The temporal experiment accumulates tokens under a single unchanged secret permutation. Its horizontal axis shows the accumulated token count. The recorded curves use two ranking settings, labeled top-10 and top-100. A reproduction retains the corresponding candidate-selection and scoring rules together with the corpus split, tokenizer, and aggregation interval.

Aldarrab and May [2] decode frequency-ranked substitution ciphertext at the character level and evaluate symbol error rate. Our SDA evaluation reports BLEU-4 for recovered text, while TFMA reports token matching. The corpus, tokenizer, decoder configuration, and scoring rules specify the adaptation to our token sequences.

J.5 Recovery Metrics

For a scored set of token positions $\mathcal{I}$, token recovery is the fraction whose reconstructed IDs equal the original IDs, TTRSR = $\frac{\sum_{i\in\mathcal{I}} \mathbf{1}\{\widehat{x}_i=x_i\}}{|\mathcal{I}|}$. A vocabulary-mapping evaluation instead defines its scored units as vocabulary IDs. The experiment's choice of scored units and aggregation across requests accompanies the reported value.

PIIRSR compares recovered personal-information items with the reference items in the privacy-evaluation corpus. Its matching rules determine the treatment of formatting variants and partially recovered entities. For recovered-text cosine similarity, the same text encoder is applied to the recovered and reference strings before computing their normalized inner product. The encoder, pooling method, PII matching rule, and dataset split are part of the reproducibility record. These details determine how an attack output becomes a table entry.